

DevOps

TROUBLESHOOTING

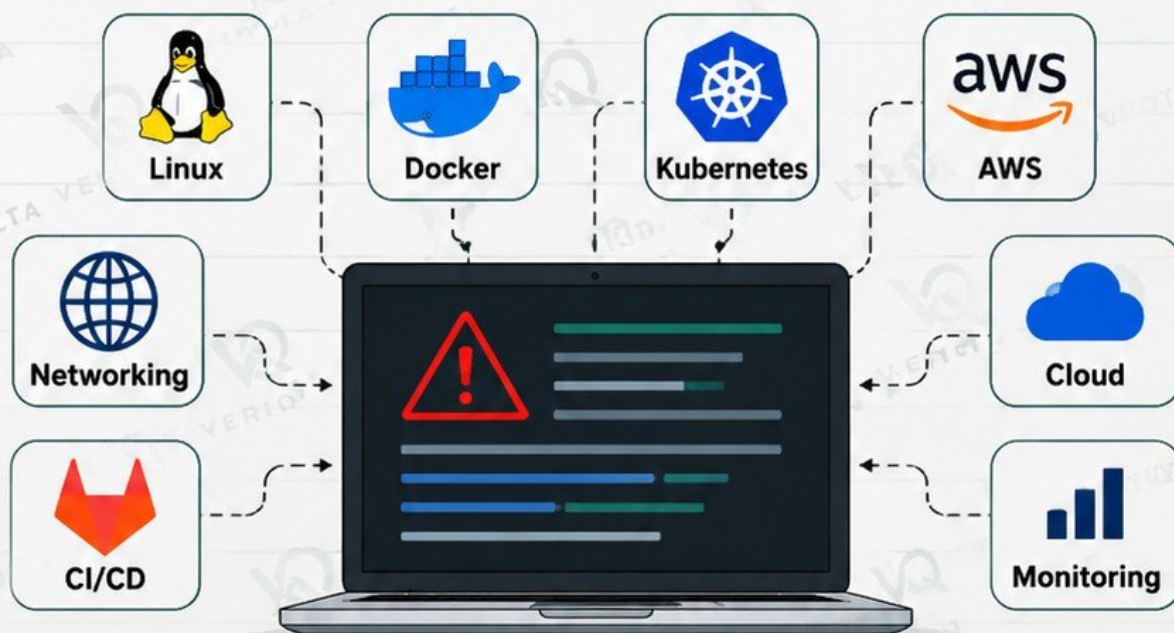
ENCYCLOPEDIA

INVESTIGATE

DEBUG

FIX

PREVENT



REAL
INCIDENTS



COMMANDS
& EXAMPLES



TROUBLESHOOTING
METHODS



PRODUCTION
SOLUTIONS



@veriqta



@veriqta



@veriqta



@veriqta



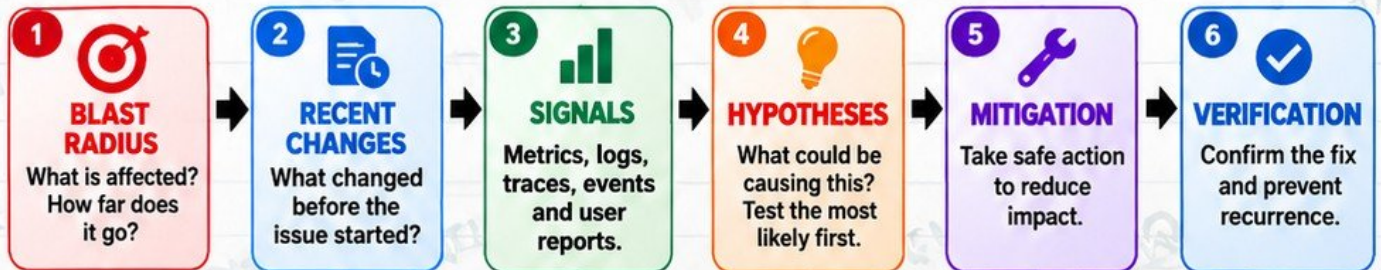
@veriqta



@veriqta

Production Troubleshooting Framework

Blast radius → Recent changes → Signals → Hypotheses → Mitigation → Verification



1 DEFINE THE BLAST RADIUS

- Identify what is impacted (users, services, regions).
- Check if it is partial or a full outage.
- Map dependent services.
- Understand customer impact and severity.

KEY QUESTIONS

- Which services are affected?
- Is it only one environment or multiple?
- Is it region specific?
- What is the business impact?

2 CHECK RECENT CHANGES

- Look at deployments, config changes, infrastructure changes, scaling events, certificate renewals.
- Check commit history and pipeline runs.
- Review change management records.

EXAMPLES

- New deployment
- Terraform apply
- Config change
- Scaling or autoscaling event
- Certificate rotation

3 GATHER SIGNALS

- Check metrics (CPU, memory, latency, error rate).
- Check logs (application, system, container, platform).
- Check traces for dependency failures.
- Check events (Kubernetes, cloud, CI/CD).

USE THESE SOURCES

- Monitoring dashboards (Prometheus, CloudWatch)
- Logs (journald, container logs, application logs)
- Traces (Jaeger, X-Ray, OpenTelemetry)
- Events (kubectl events, cloud events)
- User reports and synthetic monitors

4 FORM AND TEST HYPOTHESES

- List possible causes based on the evidence.
- Start with the highest probability.
- Test one hypothesis at a time.
- Narrow the failure domain using data.

COMMON HYPOTHESES

- Resource exhaustion
- Dependency failure
- Configuration error
- Network or DNS issue
- Bug in recent deployment

5 MITIGATE THE ISSUE

- Take the safest action to reduce user impact.
- Use rollback, restart, scale, failover or traffic shift.
- Communicate clearly with stakeholders.
- Avoid making untested changes during an incident.

EXAMPLES

- Rollback to previous version
- Scale up or down
- Restart failed components
- Failover to another AZ or region
- Disable a non critical feature

6 VERIFY AND PREVENT RECURRENCE

- Confirm the issue is fully resolved.
- Check metrics and logs for stability.
- Validate user experience.
- Document root cause and create action items.

PREVENTION STEPS

- Add better monitoring and alerting
- Improve automation
- Add tests or validation
- Update runbooks
- Address the root cause permanently



SENIOR ENGINEER NOTE

Troubleshooting is not about running random commands. It is about reducing uncertainty. Use evidence, narrow the failure domain, take safe actions, and always validate the fix.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Triage: What Is Actually Broken?

User impact, scope, dependency boundaries, symptom vs root cause

1 CHECK USER IMPACT

- Are users completely blocked or partially affected?
- Is it global, regional or specific tenants?
- What features are impacted?
- When did it start?
- Is the issue still ongoing?



No impact
Degraded
Partial outage
Full outage

2 DEFINE THE SCOPE

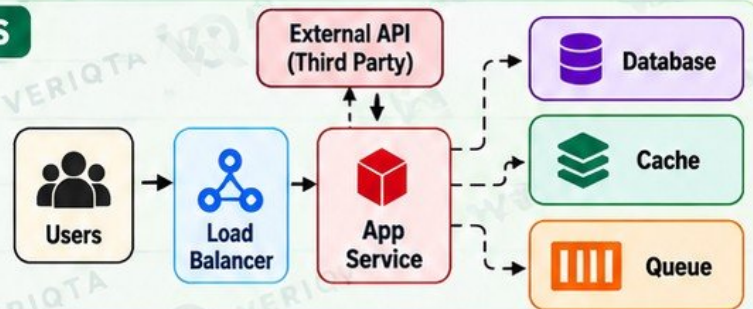
- Which environments are affected (prod, staging, dev)?
- Is it one service or multiple services?
- Is it one AZ/region or all regions?
- Is it customer facing or internal?
- Has the impact increased or stabilized?



Single service
Multiple services
Single region
Multiple regions
Global

3 IDENTIFY DEPENDENCY BOUNDARIES

- Map upstream and downstream dependencies.
- Check external services (DNS, third-party APIs, databases, queue, cloud services).
- Identify shared components.
- Understand failure domains and blast radius.



4 SYMPTOM vs ROOT CAUSE

Symptom (What you see)	Possible Root Cause (What's really wrong)
HTTP 503 errors	Downstream service or dependency failure
High latency	Network issue, resource saturation, slow query
Pod CrashLoopBackOff	Misconfiguration, missing dependency, OOM
Connection timeout	Network, security rule, DNS, or service down
High CPU usage	Traffic spike, infinite loop, inefficient code
Database slow	Locking, high load, missing index
Service unreachable	DNS failure, routing, firewall, load balancer issue



KEY QUESTIONS

- What is the exact symptom?
- What changed before it started?
- Which users are affected?
- Is it one dependency or many?
- What is the blast radius?
- What is the most likely root cause?
- What evidence can confirm or reject the hypothesis?
- What can I do safely right now?

5 TRIAGE WORKFLOW



6 PRACTICAL COMMANDS / QUERIES

- Kubernetes: `kubectl get pods -A` | `kubectl get events -A` | `kubectl describe pod <pod>`
- Logs: `kubectl logs <pod>` | `journalctl -u <service>` | `tail -n 100 /var/log/<app>.log`
- Network: `ping` | `dig` | `nslookup` | `curl -v` | `traceroute`
- AWS: `aws sts get-caller-identity` | `aws ec2 describe-instances` | `aws elbv2 describe-target-health`
- Metrics: check in Prometheus / CloudWatch / Grafana (CPU, memory, latency, error rate)



COMMON MISTAKES

- Jumping to solutions without understanding the scope.
- Focusing only on one service.
- Ignoring dependencies.
- Assuming the symptom is the root cause.
- Making untested changes in production.
- Not communicating impact clearly.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Metrics, Logs, Traces & Events

Choosing evidence sources; correlation and observability gaps

1 WHAT EACH SOURCE TELLS YOU

Metrics

- Numeric data over time
- Shows trends and anomalies
- Best for detecting issues
- Answer: How much? How many? How is it performing?

Logs

- Detailed text records
- Shows what happened
- Best for understanding cause
- Answer: What exactly happened?

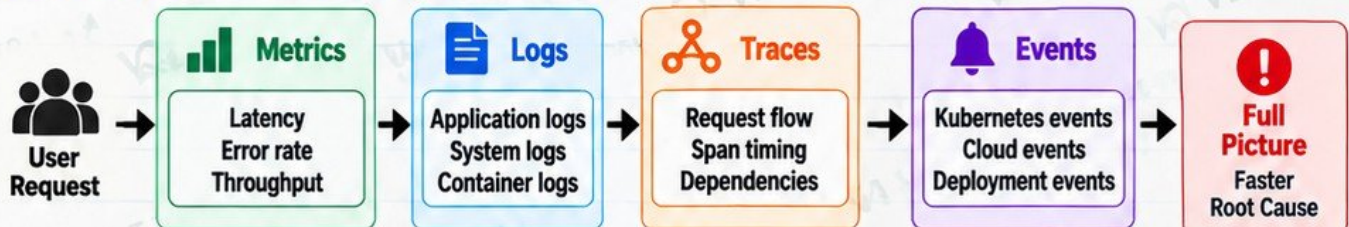
Traces

- Request flow across services
- Shows latency and bottlenecks
- Best for dependency issues
- Answer: Where did the request go? How long did each step take?

Events

- System-generated records
- Shows state changes
- Best for infrastructure and platform issues
- Answer: What changed? When did it happen?

2 HOW THEY WORK TOGETHER



3 WHEN TO USE WHICH

Scenario	Best Source	Reason
High latency	Metrics + Traces	See the impact and find the slow component
Application error	Logs + Traces	See the exact error and the request flow
Pod crashing	Logs + Events	Get error logs and container/Kubernetes events
Service unavailable	Metrics + Events	View error rate and platform state changes
Dependency failure	Traces + Logs	Find which dependency is failing
Resource saturation	Metrics + Events	See CPU/memory/disk and node events
Configuration change	Events + Logs	See what changed and what happened after

4 CORRELATION STRATEGY

- Use the same timeframe across all sources.
- Use a unique request ID (trace ID) to connect logs and traces.
- Start with metrics to detect the issue.
- Use traces to find the slow or failing component.
- Use logs to understand the exact error.
- Use events to confirm infrastructure or platform changes.
- Build dashboards that link metrics, logs, traces and events.

5 COMMON OBSERVABILITY GAPS

- Metrics without logs (alert but no context).
- Logs without traces (no request flow).
- Missing logs from containers or sidecars.
- No tracing between services (broken trace context).
- Incorrect time synchronization (clock drift).
- High cardinality metrics causing missing data.
- Events not collected or not retained.

6 EXAMPLE COMMANDS

```

# Kubernetes logs
kubectl logs -n <ns> <pod> --tail=100

# Kubernetes events
kubectl get events -n <ns> --sort-by=.lastTimestamp

# Check metrics with Prometheus (example)
rate(http_requests_total[5m])

# Find traces in Jaeger (UI or API)
Search by trace ID or service name

# Check system logs (Linux)
journalctl -u <service> --since "1 hour ago"
  
```

7 BEST PRACTICES

- Collect metrics, logs, traces and events for all services.
- Include a trace ID in all logs.
- Use centralized logging and tracing.
- Set meaningful retention periods.
- Monitor the observability stack itself.
- Ensure time is synchronized (NTP).
- Avoid missing logs from init containers and sidecars.
- Reduce metric cardinality.



SENIOR ENGINEER NOTE

No single source tells the full story. Metrics tell you there is a problem, traces show where it is, logs explain why it happened, and events confirm what changed.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Latency, Errors, Traffic & Saturation

RED/USE signals, bottleneck identification, SLO impact

1 RED METRICS (Request level)

- **Rate:** Number of requests per second.
- **Errors:** Failed requests (4xx, 5xx).
- **Duration (Latency):** Time to serve a request.
- **Best for:** User facing services and APIs.

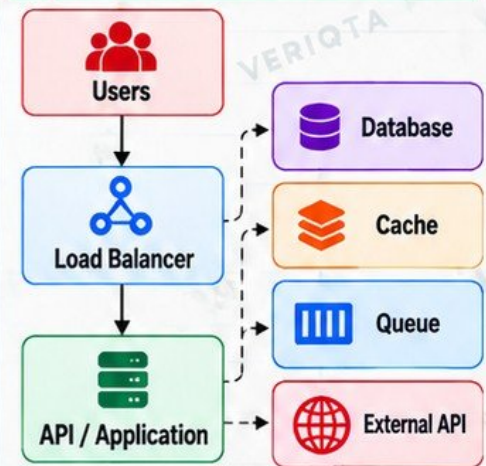
2 USE METRICS (Resource level)

- **Utilization:** How much of the resource is used (CPU, memory, disk, network).
- **Saturation:** Demand versus capacity (queue length, connections, I/O wait).
- **Errors:** Resource level failures (OOM, disk full, throttling).
- **Best for:** Infrastructure, nodes, databases, queues, etc.

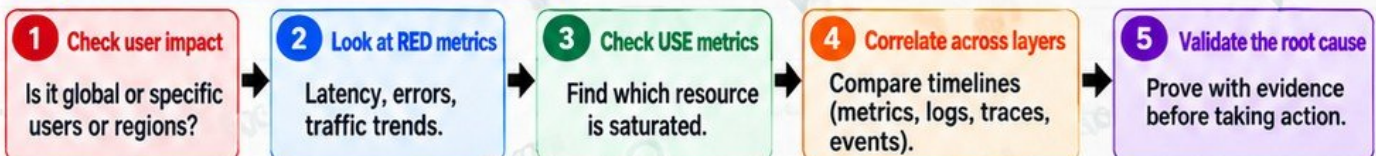
3 KEY SIGNALS AND WHAT THEY TELL YOU

Signal	What it means	Common causes
High latency	Service is slow	CPU, database, network, external dependency
High error rate	Requests are failing	Application error, dependency failure, timeout
High traffic	Increased load	Traffic spike, bot traffic, retry storm
High CPU usage	Compute is saturated	High traffic, inefficient code, noisy neighbour
High memory usage	Memory pressure	Memory leak, large cache, insufficient limits
High disk I/O	Storage bottleneck	High reads/writes, slow disk, I/O wait
High network usage	Network saturation	Large payloads, traffic spike, network issue
High queue length	Requests are waiting	Downstream slowness, insufficient workers
High connection count	Too many connections	Connection leaks, missing pooling, slow responses

4 TYPICAL ARCHITECTURE VIEW



5 IDENTIFYING THE BOTTLENECK



6 EXAMPLE METRICS (INTERPRETATION)

Metric	Normal	Potential issue
API latency (p95)	< 200 ms	> 1 s (check dependencies)
Error rate	< 1%	> 5% (check logs and recent changes)
Request rate	Normal pattern	Sudden spike (check source)
CPU usage	< 70%	> 90% (CPU saturation)
Memory usage	< 70%	> 90% (memory pressure)
Disk I/O wait	< 5%	> 20% (storage bottleneck)
Queue length	Near 0	Growing consistently (downstream issue)

7 SLO IMPACT

- Latency increases can burn your error budget.
- High error rate directly impacts availability SLOs.
- Sustained traffic spikes can cause cascading failures.
- Identify which SLI is affected (latency, error rate, availability).
- Use SLOs to prioritize and decide on mitigation.

8 COMMON TOOLS

- **Metrics:** Prometheus, Grafana, CloudWatch
- **Logs:** journalctl, Azure Logs, CloudWatch Logs
- **Traces:** Jaeger, OpenTelemetry
- **Events:** kubectl events, cloud events, deployment events
- **Dashboards:** Correlate all signals on a single timeline



SENIOR ENGINEER NOTE

A high metric does not always mean it is the root cause. Always correlate metrics, logs, traces and events, check recent changes, and validate before taking action.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

CPU Saturation & High Load

Load average, run queue, steal time, throttling, top, pidstat, vmstat

1 KEY CONCEPTS

- **CPU saturation** means the CPU cannot keep up with the work being requested.
- **Load average** shows the average number of runnable or uninterruptible tasks.
- **Run queue** is the number of processes waiting for CPU time.
- **Steal time** is CPU time taken by the hypervisor (common in cloud VMs).
- **CPU throttling** happens when a process or container exceeds its CPU limit (CFS throttling).

2 UNDERSTANDING LOAD AVERAGE

- Load average represents tasks in run queue or waiting for I/O (not just CPU usage).
- Compare load average with the number of CPU cores.
- Sustained load average higher than number of cores indicates CPU contention.

Load Average	System State	Interpretation
< number of cores	Healthy	CPU can handle the load
≈ number of cores	Busy	Near saturation
> number of cores	Saturated	Tasks are waiting in queue

3 IMPORTANT COMMANDS

```
# Check load average and CPU usage
uptime                # Load average, running tasks
```

```
# Real-time CPU usage per process
top                   # Press 1 for per-CPU view
```

```
# Detailed per-process metrics
pidstat -u 1 5        # CPU usage per process
```

```
# CPU, run queue, context switch, I/O wait
vmstat 1 5            # Overall system view
```

```
# Per-CPU usage
mpstat -P ALL 1 5     # CPU usage per core
```

4 INTERPRETING vmstat OUTPUT

```
vmstat 1 5
```

procs		memory			swap	io		system		cpu						
r	b	swpd	free	buff	cache	si	so	bi	bo	in	cs	us	sy	id	wa	st
4	1	0	512M	120M	2.1G	0	0	12	50	3000	5000	70	20	5	3	2

- **r** Run queue (runnable tasks waiting for CPU).
- **b** Blocked tasks (usually waiting for I/O).
- **us** User CPU time (application code).
- **sy** System CPU time (kernel).
- **wa** I/O wait (waiting for disk or network).
- **st** Steal time (CPU taken by hypervisor). High value indicates noisy neighbours or insufficient vCPU.

5 COMMON CAUSES

- High traffic or sudden load spike.
- Inefficient or CPU-intensive code.
- Infinite loops or runaway processes.
- Insufficient CPU requests/limits (containers).
- Noisy neighbour on shared infrastructure.
- High system processes (e.g. backup, scanning).
- Excessive logging or encryption overhead.

6 HOW TO INVESTIGATE

- 1 Check load average (uptime).
- 2 Identify high CPU processes (top, pidstat).
- 3 Check per-core usage (mpstat).
- 4 Look for CPU throttling (if using containers).
- 5 Check for steal time (vmstat).
- 6 Correlate with application logs and metrics.
- 7 Identify recent changes or deployments.
- 8 Narrow the root cause and mitigate.

8 SIGNS OF CPU SATURATION

- High load average for sustained period.
- High %us and %sy in vmstat or top.
- Increased response time and errors.
- Processes stuck in run queue (high r).
- Container CPU throttling events.

9 MITIGATION STEPS

- Scale up (more CPU) or scale out (more instances).
- Optimize CPU-intensive code.
- Set appropriate CPU requests and limits.
- Reduce unnecessary background processes.
- Use profiling to find the hot code path.
- Implement caching or async processing.

7 CONTAINER CPU THROTTLING

Check if a container is being throttled:

```
kubectl describe pod <pod>
# Look for CPU Throttling events
```

```
kubectl top pod <pod>
# Check CPU usage
```

```
cat /sys/fs/cgroup/cpu/cpu.stat
# nr_throttled: number of times
# throttled_time: total time (ns)
```

10 BEST PRACTICES

- Set realistic CPU requests and limits.
- Monitor load average, run queue and throttling.
- Use autoscaling based on CPU and latency.
- Optimize code and avoid busy loops.
- Use profiling (pprof, flame graphs).
- Correlate CPU metrics with application metrics.
- Alert before saturation impacts users.



SENIOR ENGINEER NOTE

High CPU usage is a symptom. Always correlate CPU metrics with application metrics, logs, traces and recent changes to find the real cause.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Linux Memory Pressure

RSS, page cache, swap, reclaim, OOM killer, cgroups

1 KEY CONCEPTS

- **RSS:** Actual physical memory used by a process.
- **Page cache:** Memory used to cache filesystem data.
- **Swap:** Disk space used when RAM is exhausted.
- **Reclaim:** Kernel process to free unused memory.
- **OOM killer:** Kills processes when the system is out of memory.
- **cgroups:** Limits and controls memory usage for processes/containers.

3 CHECK MEMORY USAGE

```
# Overall memory usage
free -h
```

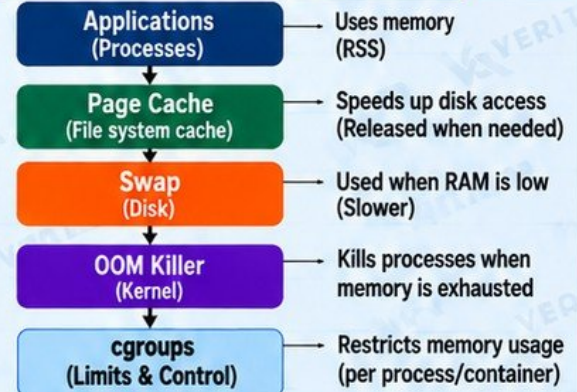
```
# Detailed memory information
cat /proc/meminfo
```

```
# Real-time memory usage
watch -n 1 free -h
```

```
# Active processes sorted by memory usage
top
```

```
# Per-process memory details
ps aux --sort=-%mem | head -n 10
```

2 LINUX MEMORY ARCHITECTURE



4 INTERPRETING free -h OUTPUT

Field	Meaning	What to look for
total	Total RAM	Baseline capacity
used	Used memory	High value indicates pressure
free	Unused memory	Low value may be normal (Linux uses cache)
buff/cache	Page cache + buffers	Reclaimable memory
available	Memory available for new processes	Low value indicates memory pressure
swap	Swap space	High usage indicates memory shortage

5 COMMON MEMORY ISSUES

- High memory usage (RSS) by applications.
- Page cache consuming large memory.
- Swap usage increasing.
- Memory not being reclaimed.
- OOM killer terminating processes.
- cgroup memory limit being reached.
- Memory leak in applications.

6 CHECK SWAP USAGE

```
# Check swap usage
swapon -s
free -h
```

```
# Monitor swap activity
vmstat 1 5
```

What to look for:

- Consistently high swap usage.
- High si (swap in) and so (swap out) in vmstat.
- Applications frequently swapping (may indicate memory shortage).

7 CHECK OOM KILLER EVENTS

```
# View OOM killer logs
journalctl -k | grep -i 'oom'
```

```
# Check kernel messages
dmesg | grep -i 'killed process'
```

What to look for:

- OOM killer messages.
- Which process was killed.
- Reason for termination (out of memory).
- Frequency of OOM events.

8 CHECK CGROUP MEMORY USAGE

```
# For cgroup v1
cat /sys/fs/cgroup/memory/memory.usage_in_bytes
cat /sys/fs/cgroup/memory/memory.limit_in_bytes
```

```
# For cgroup v2
cat /sys/fs/cgroup/memory.current
cat /sys/fs/cgroup/memory.max
```

What to look for:

- Current memory usage.
- Memory limit.
- Whether the process/container is hitting the limit.
- OOM events in cgroup memory.events.

9 MITIGATION STEPS

- Identify high memory processes and optimize.
- Increase memory limits (node or container).
- Reduce memory leaks in applications.
- Tune application memory settings (JVM, etc.).
- Use proper requests and limits in Kubernetes.
- Monitor and alert on memory usage and swap.

10 USEFUL COMMANDS SUMMARY

Purpose	Command
Memory usage overview	free -h
Detailed memory info	cat /proc/meminfo
Real-time stats	vmstat 1 5
Top memory processes	top or ps aux --sort=-%mem
Check swap usage	swapon -s
Check OOM killer logs	journalctl -k grep -i 'oom'
Check cgroup memory (v2)	cat /sys/fs/cgroup/memory.current



SENIOR ENGINEER NOTE

Low free memory is not always a problem. Focus on available memory, swap activity, OOM events, and application behaviour to determine real memory pressure.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Disk I/O & Filesystem Failures

IOPS, latency, iowait, inode exhaustion, filesystem-full incidents

1 KEY CONCEPTS

- **Disk I/O:** Read and write operations to storage devices.
- **IOPS:** Number of input/output operations per second.
- **Latency:** Time taken to complete a read or write.
- **iowait:** CPU time waiting for disk I/O to complete.
- **Inode exhaustion:** No more inodes available to create files.
- **Filesystem full:** No more space available on the filesystem.
- **Common causes:** High I/O, slow storage, large files, log growth, application issues, metadata/inode exhaustion.

2 DISK I/O METRICS EXPLAINED

Metric	Meaning	High value indicates
IOPS	I/O operations per second	High load on disk
Latency (await)	Time for I/O to complete (ms)	Slow storage or saturation
iowait	CPU time waiting for I/O	Disk bottleneck
r/s / w/s	Read/Write operations per second	High I/O activity
rkB/s / wkB/s	Read/Write throughput (KB/s)	Large data being read/written
%util	Percentage of time device is busy	Near 100% = saturated
Inodes	Number of file entries	High usage = cannot create new files

3 CHECK DISK I/O USAGE

```
# Real-time disk I/O statistics
iostat -xz 1

# Detailed view with device names
iostat -xm 1

# Disk I/O using vmstat
vmstat 1 5

# Per-process I/O usage
pidstat -d 1 5

# Real-time disk activity
iotop -o
```

4 INTERPRETING iostat OUTPUT

Field	Meaning	What to look for
r/s	Reads per second	Sudden increase
w/s	Writes per second	High write activity
rkB/s	Read throughput	Large reads
wkB/s	Write throughput	Large writes
await	I/O latency (ms)	> 10-20 ms indicates slow disk
svctm	Service time (ms)	High value = slow device
%util	Device utilization	> 70-80% = near saturation

5 CHECK iowait AND CPU

```
# Check CPU and iowait
top
# Look at %wa (iowait)

# Alternative view
mpstat -P ALL 1 5
# Look at %iowait

# System wide stats
vmstat 1 5
# Look at wa column
```

6 CHECK FILESYSTEM SPACE

```
# Check disk space usage
df -h

# Check inode usage
df -i

# Detailed filesystem information
lsblk -f

# Find large directories
du -sh /* | sort -h

# Find large files (top 10)
find / -type f -size +1G -exec ls -lh {} \;
2>/dev/null | sort -k 5 -hr | head -10
```

7 INTERPRETING df OUTPUT

Field	Meaning	Issue
Filesystem	Device/mount point	Identify partition
Size	Total size	-
Used	Used space	High value
Avail	Free space	Low value = filesystem full
Use%	Percentage used	> 80% = warning > 90% = critical
Mounted on	Mount point	Application data path

8 INODE EXHAUSTION

- Each file and directory uses an inode.
- If inodes are exhausted, you cannot create new files even if there is free disk space.
- **Common causes:** too many small files (logs, cache, temp files).
- **Check inode usage:**

```
df -i
```

What to do:

- Find and remove unnecessary files.
- Rotate or compress logs.
- Adjust application behaviour (e.g. reduce file creation).

9 FILESYSTEM-FULL INCIDENTS

- Applications fail to write logs, cache or data.
- Services may crash or become unresponsive.
- Databases and containers can stop working.
- Can lead to wider system failures.

10 TROUBLESHOOTING STEPS

- 1 Check disk space and inode usage (df -h, df -i).
- 2 Identify which partition is full.
- 3 Find large files and directories (du, find).
- 4 Check disk I/O and latency (iostat, iotop, vmstat).
- 5 Look at application logs for errors.
- 6 Clean up unused files, rotate logs or expand storage.
- 7 Monitor to ensure the issue is resolved.

11 COMMON CAUSES

- High log volume (application or system).
- Large temporary files.
- Unexpected data growth.
- Runaway container workloads.
- Database high write activity.
- Inode exhaustion (many small files).
- Slow or failing storage devices.
- Misconfigured applications.



SENIOR ENGINEER NOTE

High iowait, high latency or a full filesystem can cause application timeouts and system instability. Always monitor disk usage, inode usage and I/O latency before it impacts production.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Processes, Signals & systemd Failures

Process states, zombies, signals, service restart loops, journald

1 KEY CONCEPTS

- A **process** is a running program with a unique PID.
- **Process states** show what the process is doing.
- **Zombies** are defunct processes waiting for the parent to reap them.
- **Signals** are used to control processes (stop, terminate, reload, etc).
- **systemd** manages services, handles restarts and logs to journald.
- **Service restart loops** occur when a service keeps failing and restarting.

2 PROCESS STATES (ps STAT)

State	Meaning	Common cause
R	Running	Actively using CPU or ready to run
S	Sleeping	Waiting for an event (I/O, timer, etc)
D	Uninterruptible sleep	Waiting for I/O (cannot be killed easily)
T	Stopped	Stopped (e.g. by SIGSTOP)
Z	Zombie	Process finished but not reaped by parent
<	High priority	High priority process
N	Low priority	Low priority process
	Idle	Kernel thread (idle)

3 CHECK RUNNING PROCESSES

```
# Show all processes
ps aux --sort=-%cpu | head -10

# Interactive process viewer
top # press 1 for per-CPU, H for threads

# Modern view
htop

# Filter by process name
ps aux | grep nginx

# Show process tree
pstree -p
```

4 IDENTIFY ZOMBIE PROCESSES

```
# Find zombie processes
ps aux | awk '$8=="Z"'

# Another way
ps -eo pid,ppid,stat,cmd | grep " Z "
```

Field	Meaning
PID	Process ID
PPID	Parent process ID
STAT	Process state (Z = zombie)
CMD	Command

Why zombies happen?

- Parent process did not reap (wait) for child.
- Bug in application code.
- Parent process has died (or is stuck).

5 COMMON SIGNALS

Signal	Number	Purpose
SIGTERM	15	Graceful termination
SIGKILL	9	Force kill (cannot be ignored)
SIGINT	2	Interrupt (Ctrl+C)
SIGHUP	1	Reload configuration
SIGSTOP	19	Stop process
SIGCONT	18	Continue stopped process
SIGUSR1	10	User defined signal 1
SIGUSR2	12	User defined signal 2

Send a signal to a process

```
kill -15 <PID> # graceful stop
kill -9 <PID> # force kill
kill -1 <PID> # reload (SIGHUP)
```

6 CHECK systemd SERVICE STATUS

```
# Check service status
systemctl status <service>

# Check if service is running
systemctl is-active <service>

# Check if service is enabled (at boot)
systemctl is-enabled <service>

# List failed services
systemctl --failed
```

7 SERVICE RESTART LOOPS

```
# Check recent restarts
systemctl status <service>

# Show restart count
systemctl show <service> | grep NRestarts

# View logs for the service
journalctl -u <service> --no-pager -n 50
```

Common causes

- Application crashes (code error, missing config).
- Dependency service not available.
- Permission or file path issues.
- Resource limits (memory, file descriptors).
- Misconfigured environment variables.

8 ANALYZE LOGS WITH JOURNALD

```
# View recent logs
journalctl -u <service> -n 50

# Follow logs in real time
journalctl -u <service> -f

# Filter by time range
journalctl -u <service> --since "1 hour ago"

# View only errors
journalctl -u <service> -p err

# Logs for a specific boot
journalctl -u <service> -b
```

9 TROUBLESHOOTING FLOW

- 1 Check if the process or service is running.
- 2 Look at process state (R, S, D, Z, etc).
- 3 Identify high resource usage (CPU, memory, I/O).
- 4 Check for zombies and their parent process.
- 5 Inspect service status and restart count.
- 6 Analyze journald logs for errors.
- 7 Fix the root cause and monitor.

10 HELPFUL COMMANDS SUMMARY

Purpose	Command
List processes	ps aux
Real-time view	top / htop
Process tree	ps tree -p
Find zombies	ps aux awk '\$8=="Z"'
Send signal	kill -15 <PID>
Service status	systemctl status <service>
Restart service	systemctl restart <service>
View service logs	journalctl -u <service> -n 50
Filter error logs	journalctl -u <service> -p err

11 BEST PRACTICES

- Use proper signal (SIGTERM) for graceful shutdown.
- Fix the root cause instead of just restarting services.
- Set resource limits and monitoring.
- Review and rotate logs regularly.
- Use systemd restart policies wisely (Restart=on-failure).
- Alert on frequent restarts or crashes.

★ SENIOR ENGINEER NOTE

Processes, signals and systemd failures are often symptoms. Always correlate with system resources, logs and dependencies to find the real cause.

File Descriptor & Socket Exhaustion

ulimit, FD leaks, connection exhaustion, kernel limits

1 KEY CONCEPTS

- **File descriptor (FD):** A handle to an open file, socket, or resource.
- Each process has a limit on the number of open file descriptors.
- **FD leaks** happen when applications do not close files or sockets.
- **Connection exhaustion** occurs when all available sockets/ports are in use.
- **Kernel limits** (somaxconn, ip_local_port_range, file-max) control system-wide resources.

2 COMMON ERROR MESSAGES

- Too many open files
- **EMFILE** (per-process limit reached)
- **ENFILE** (system-wide limit reached)
- Cannot assign requested address
- Address already in use
- No buffer space available
- Connection refused (due to port/socket exhaustion)
- Resource temporarily unavailable

3 CHECK FILE DESCRIPTOR USAGE

```
# Check per-process open file descriptors
lsof -p <PID> | wc -l
```

```
# List open files and sockets
lsof -p <PID>
```

```
# Show all open files system-wide
lsof | wc -l
```

```
# Check file descriptor limit for current shell
ulimit -n
```

```
# Check process limits
cat /proc/<PID>/limits | grep "open files"
```

4 UNDERSTANDING ulimit

Command	Meaning	Example Output	What it means
ulimit -n	Max open files for current process	1024	Per-process file descriptor limit
ulimit -Hn	Hard limit	65536	Maximum allowed
ulimit -Sn	Soft limit	1024	Current limit (can be increased up to hard limit)

6 KERNEL LIMITS AND CONFIGURATION

Parameter	Meaning	Default (varies)	What it controls
fs.file-max	Total file handles in system	~9223372036854775807	System-wide open file limit
fs.nr_open	Max per-process file descriptors	1048576	Upper limit for ulimit
net.ipv4.ip_local_port_range	Ephemeral port range	32768 60999	Ports used for outgoing connections
net.core.somaxconn	Max pending connections	4096	Queue length for incoming connections
net.ipv4.tcp_max_syn_backlog	Max SYN queue size	4096	Half-open connections
net.netfilter.nf_conntrack_max	Max tracked connections	65536	Connection tracking limit

5 CHECK SOCKET AND CONNECTION USAGE

```
# Check established connections
ss -ant | head -20
```

```
# Check connection count by state
ss -s
```

```
# Show connections to a specific port
ss -ant | grep :80
```

```
# Count connections by state
ss -ant | awk '{print $1}' | sort | uniq -c
```

```
# Check for many connections from same IP
ss -ant | awk '{print $5}' | cut -d:-f1 | sort | uniq -o+ | sort -nr | head -10
```

7 IDENTIFYING FD LEAKS

- Monitor open file descriptors over time (lsof, /proc).
- Look for a continuous increase in FD count.
- Check application logs for warnings or errors.
- Identify code paths that do not close files or sockets.
- Use tools like lsof, strace or application metrics.

8 TROUBLESHOOTING STEPS

- 1 Check current FD and connection usage.
- 2 Identify which process is using most FDs/sockets.
- 3 Compare with ulimit and kernel limits.
- 4 Look for FD leaks in application code or libraries.
- 5 Check for connection exhaustion (high CLOSE_WAIT or TIME_WAIT).
- 6 Increase limits if needed.
- 7 Restart affected services after fixing the issue.

9 COMMON CAUSES

- Application not closing files or sockets.
- High number of concurrent connections.
- Improper connection pooling.
- Short-lived connections (no keep-alive).
- Insufficient ulimit or kernel limits.
- Sudden traffic spikes.
- Misconfigured load balancers or proxies.

10 BEST PRACTICES

- Set appropriate ulimit for applications.
- Tune kernel parameters based on workload.
- Use connection pooling and keep-alive.
- Monitor FD and connection metrics.
- Set alerts for high FD usage and connection counts.
- Regularly review and rotate logs.



SENIOR ENGINEER NOTE

File descriptor and socket exhaustion can cause application failures, connection drops and system instability. Always monitor usage and set proper limits.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

DNS Failure Investigation

Resolution path, caching, TTL, SERVFAIL, NXDOMAIN, resolver failures

1 KEY CONCEPTS

- DNS translates domain names to IP addresses.
- **Resolution path:** Client → Resolver → Root → TLD → Authoritative → IP address.
- **Caching** stores DNS responses for a period defined by TTL.
- **TTL (Time To Live)** controls how long a record is cached.
- **SERVFAIL** indicates a server-side DNS error.
- **NXDOMAIN** means the domain does not exist.
- **Resolver failures** occur when the DNS resolver cannot reach upstream servers or returns errors.

3 COMMON DNS COMMANDS

```
# Query DNS record
dig example.com

# Short output (IP only)
dig +short example.com

# Detailed query with trace (resolution path)
dig +trace example.com

# Check using nslookup
nslookup example.com

# Check DNS from specific server
dig @8.8.8.8 example.com

# Check DNS cache (systemd-resolved)
resolvectl cache

# Show current DNS configuration
cat /etc/resolv.conf
```

5 CHECK CACHING AND TTL

```
# Check TTL value
dig example.com | grep TTL

# Show full answer section
dig example.com

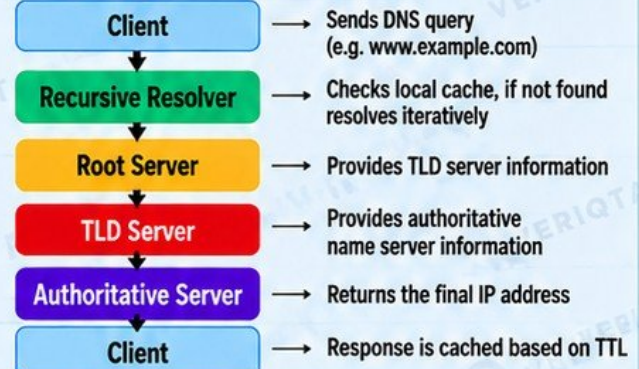
# Check resolver cache (systemd)
resolvectl query example.com

# Flush cache (systemd)
sudo resolvectl flush-caches
```

Cache behaviour:

- Results are cached based on TTL.
- Lower TTL = faster changes but more queries.
- Higher TTL = less load but slower propagation.
- You may need to flush local cache when testing changes.

2 DNS RESOLUTION PATH



4 COMMON DNS ERRORS

Error	Meaning	Common Cause	What to Check
SERVFAIL	Server failed to complete the request	Authoritative or upstream server issue	Check authoritative server, logs, DNSSEC
NXDOMAIN	Domain does not exist	Wrong domain name or not created	Check spelling, registration, zone file
TIMEOUT	No response from DNS server	Network issue, firewall, UDP/TCP blocked	Check connectivity, firewall, port 53
REFUSED	Query refused	Access control, misconfiguration	Check server ACLs and settings
FORMERR	Malformed query	Client or server bug	Check query format and DNS software

6 CHECK RESOLVER FAILURES

```
# Check resolver status
resolvectl status

# Test DNS server connectivity
dig @8.8.8.8 example.com

# Check if port 53 is reachable (UDP)
nc -u -zv 8.8.8.8 53

# Check if port 53 is reachable (TCP)
nc -zv 8.8.8.8 53

# Check system DNS configuration
cat /etc/systemd/resolved.conf
```

Common resolver issues:

- Wrong DNS server configured.
- Network or firewall blocking port 53.
- Resolver service not running.
- Upstream DNS server down.
- Misconfigured DNS search domains.
- VPN or network policy interfering.

7 TROUBLESHOOTING FLOW

- 1 Verify the exact error (SERVFAIL, NXDOMAIN, timeout).
- 2 Check local resolver and cache.
- 3 Test with public DNS (e.g. 8.8.8.8, 1.1.1.1).
- 4 Use dig +trace to follow the resolution path.
- 5 Verify network connectivity and firewall rules.
- 6 Check authoritative DNS server and zone configuration.
- 7 Review TTL and allow time for propagation.

8 COMMON CAUSES

- Incorrect domain name.
- DNS server outage.
- Network or firewall blocking DNS.
- Misconfigured resolver settings.
- Expired domain or missing records.
- DNSSEC issues.
- High TTL causing stale records.
- Load balancer or CDN DNS misconfiguration.
- Internal DNS zone misconfiguration.

9 BEST PRACTICES

- Use multiple DNS servers for redundancy.
- Monitor DNS availability and latency.
- Set appropriate TTL values.
- Implement alerting for DNS failures.
- Validate DNS changes before production.
- Keep resolver software updated.
- Use DNS health checks in critical systems.
- Document DNS architecture and dependencies.



SENIOR ENGINEER NOTE

DNS issues can cause complete application outages. Always check from multiple locations, verify the entire resolution path, and correlate with network, firewall and application logs.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

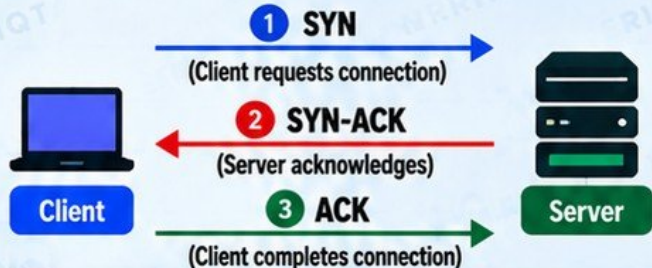
TCP Connection Failures

SYN/SYN-ACK, resets, retransmissions, TCP states, TIME_WAIT

1 KEY CONCEPTS

- TCP uses a three-way handshake: SYN → SYN-ACK → ACK.
- Connection failures occur when the handshake does not complete.
- Resets (RST) indicate a connection was actively rejected.
- Retransmissions happen when packets are not acknowledged.
- TCP states show the current status of a connection.
- TIME_WAIT keeps closed connections for a period to prevent old packets from interfering with new connections.

2 TCP THREE-WAY HANDSHAKE



3 COMMON TCP COMMANDS

```
# Check open TCP connections
ss -ant
```

```
# Show TCP connection states (summary)
ss -s
```

```
# Check connections to a specific host and port
ss -ant | grep :80
```

```
# Real-time connection view
watch -n 1 'ss -ant | head -n 20'
```

```
# Capture TCP traffic (handshake, resets, retransmits)
tcpdump -nn 'tcp port 80'
```

```
# Show detailed connection info
netstat -antp | grep :80
```

4 IMPORTANT TCP STATES

State	Meaning	Common Cause / Notes
SYN_SENT	SYN sent, waiting for SYN-ACK	Server down, firewall blocking, network issue
SYN_RECV	SYN received, waiting for ACK	High load, SYN flood, or client not completing handshake
ESTABLISHED	Connection is active	Normal state
FIN_WAIT1	Waiting for FIN from peer	Normal during connection close
FIN_WAIT2	FIN sent, waiting for FIN from peer	Peer not closing connection
TIME_WAIT	Connection closed, waiting to clear	Normal, prevents old packets from affecting new connections
CLOSE_WAIT	Peer closed, waiting for local close	Application not closing connection
LAST_ACK	Waiting for final ACK	Normal during close process
CLOSED	No connection	Normal state

5 COMMON TCP ERRORS

Error / Symptom	Meaning	Common Cause
Connection refused	RST received	Service not running or port closed
Connection timed out	No response	Firewall, network issue, or server down
No route to host	Network unreachable	Routing or network configuration issue
SYN retransmissions	No SYN-ACK	Firewall blocking or server not reachable
High TIME_WAIT	Many closed connections	Normal with high traffic or short-lived connections
Connection reset	RST from server	Application closed or rejected connection

6 CHECK FOR RETRANSMISSIONS

```
# Capture and analyze retransmissions
tcpdump -nn -i any 'tcp[tcpflags] & (tcp-syn|tcp-ack) != 0'
```

```
# View retransmissions in real time
ss -ti | grep retrans
```

```
# Check packet loss and retransmits (summary)
netstat -s | grep -i retrans
```

What it means:

- Retransmissions occur when packets are not acknowledged.
- High retransmissions indicate network issues, packet loss or high latency.
- Can lead to slow connections or failures.

7 TROUBLESHOOTING FLOW

- Check if the service is running (server and port).
- Test connectivity (ping, traceroute).
- Check firewall and security group rules.
- Use ss or netstat to check connection state.
- Look for SYN retransmissions or RST packets.
- Check server logs and application logs.
- Investigate high TIME_WAIT or connection limits.

8 COMMON CAUSES

- Service not running or crashed.
- Firewall blocking the port.
- Network misconfiguration or routing issue.
- Server overloaded or high resource usage.
- Application actively rejecting connections.
- High number of short-lived connections (causing TIME_WAIT).
- Connection limits reached.
- Packet loss or high latency.

9 BEST PRACTICES

- Monitor connection counts and states.
- Set appropriate timeouts and retry logic.
- Tune TCP parameters for high traffic (e.g. backlog, TIME_WAIT, port range).
- Use load balancing and health checks.
- Ensure firewalls allow required traffic.
- Set proper connection limits in applications.
- Monitor for retransmissions, resets and unusual TCP states.



SENIOR ENGINEER NOTE

TCP connection failures can be caused by network, firewall, server, application or resource limits. Always check from client to server and correlate with logs and metrics.

TLS & Certificate Failures

Handshake failures, trust chains, hostname mismatch, expiry, SNI

1 KEY CONCEPTS

- **TLS** (Transport Layer Security) encrypts data between client and server.
- The **handshake** authenticates the server (and optionally client) using certificates.
- A **trust chain** links the server certificate to a trusted **Certificate Authority (CA)**.
- **Failures** can occur due to expired certificates, hostname mismatch, invalid chain, unsupported TLS version or cipher, or SNI issues.
- **SNI (Server Name Indication)** tells the server which domain you are connecting to when multiple domains share the same IP.

2 TLS HANDSHAKE FLOW



3 USEFUL COMMANDS

```

# Check certificate details and chain
openssl s_client -connect example.com:443 -showcerts

# Check certificate expiry date
openssl x509 -in cert.pem -noout -dates

# Verify certificate and full chain
openssl verify -CAfile ca-bundle.pem cert.pem

# Check TLS version and cipher
openssl s_client -connect example.com:443 -tls1_2

# Test connection with SNI (important for shared hosting)
openssl s_client -connect example.com:443 -servername example.com

# Quick check using curl
curl -Iv https://example.com
  
```

4 COMMON TLS ERRORS

Error Message	Meaning	Common Cause	What to Check
certificate expired	Certificate is no longer valid	Expired certificate	Check expiry date and renew
unable to get local issuer certificate	Cannot verify the certificate chain	Missing intermediate certificate	Ensure full chain is provided
hostname mismatch	Certificate domain does not match	Wrong domain on certificate	Check certificate CN/SAN
handshake failure	TLS handshake could not complete	Unsupported TLS version or cipher, firewall, SNI	Check TLS version, cipher and SNI
unknown ca	Certificate not trusted	Self-signed or untrusted CA	Add correct CA or use trusted CA
unrecognized name	SNI not provided or wrong	Multiple domains on same IP	Use -servername with openssl
connection reset during handshake	Connection closed during handshake	Firewall, wrong protocol, unsupported cipher	Check network, protocol and cipher

5 CHECK CERTIFICATE DETAILS

```

# View certificate details
openssl s_client -connect example.com:443 -servername example.com

# View only certificate information
openssl s_client -connect example.com:443 -servername example.com \
| openssl x509 -noout -text
  
```

6 UNDERSTAND CERTIFICATE FIELDS

Field	Meaning	Example	What to Check
Subject (CN)	Main domain name	CN=example.com	Matches your domain
Subject Alternative Name (SAN)	Additional domains	DNS:example.com	Domain is in SAN list
Issuer	Certificate Authority	Let's Encrypt	Trusted CA
Valid From	Start date	Jan 1 2025	Check validity period
Valid To	Expiry date	Jan 1 2026	Not expired
Signature Algorithm	Algorithm used	sha256WithRSA	Modern and secure
Public Key	Server public key	RSA 2048	Strong key length

7 TROUBLESHOOTING FLOW

- 1 Confirm the service is reachable (ping, nc, curl).
- 2 Check certificate expiry date.
- 3 Verify the full certificate chain.
- 4 Check for hostname mismatch (CN/SAN).
- 5 Test with the correct SNI (if using shared IP).
- 6 Check TLS version and cipher compatibility.
- 7 Look for firewall or network issues.
- 8 Review server logs for TLS or handshake errors.

8 COMMON CAUSES

- Expired certificate.
- Missing intermediate certificate.
- Hostname mismatch.
- Self-signed or untrusted certificate.
- Incorrect or missing SNI.
- Unsupported TLS version or cipher.
- Firewall or security device blocking handshake.
- Misconfigured web server (e.g. Nginx, Apache).
- Wrong domain configured on load balancer or proxy.

9 BEST PRACTICES

- Use certificates from trusted CAs.
- Keep certificates renewed before expiry.
- Install full certificate chain (including intermediates).
- Use correct domain names (CN/SAN).
- Ensure SNI is configured on clients and servers.
- Disable old TLS versions (e.g. TLS 1.0, 1.1).
- Use strong ciphers and monitor certificate expiry.
- Set up alerts for certificate expiration.

★ SENIOR ENGINEER NOTE

TLS and certificate issues can cause complete service outages. Always verify the full chain, domain name and expiry, and check network, load balancers and firewall logs if the handshake fails.

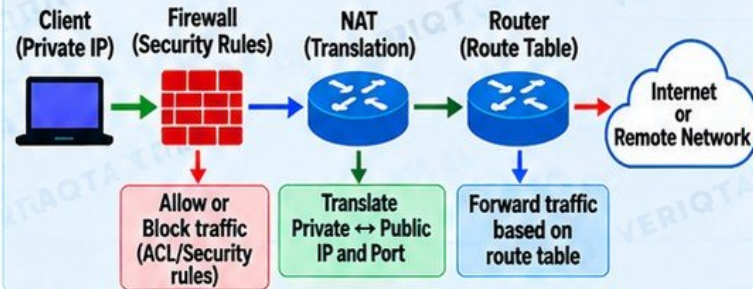
Routing, NAT & Firewall Failures

Route tables, NAT exhaustion, ACL/security rules, asymmetric routing

1 KEY CONCEPTS

- **Routing** decides the best path for traffic to reach a destination.
- **NAT (Network Address Translation)** translates private IPs to public IPs (and vice versa).
- **Firewalls, ACLs and security rules** allow or block traffic based on rules.
- **NAT exhaustion** happens when all available ports are used.
- **Asymmetric routing** occurs when the request and response take different paths, causing connectivity issues.

2 TRAFFIC FLOW OVERVIEW



3 USEFUL COMMANDS (LINUX)

```

# View routing table
ip route show

# Check route to a specific destination
ip route get 8.8.8.8

# Test connectivity and show path
traceroute 8.8.8.8

# Check NAT translation table (if using iptables)
sudo iptables -t nat -L -n -v

# Check firewall rules (iptables)
sudo iptables -L -n -v

# Check connection tracking (NAT sessions)
sudo conntrack -L | grep <IP>

# Test port connectivity
nc -zv <host> <port>

# View real-time network connections
ss -antp
  
```

4 COMMON FAILURES AND SYMPTOMS

Issue	Symptoms	Common Cause	What to Check
No route to host	Cannot reach remote network	Missing or incorrect route table entry	ip route, route table in cloud/infra
NAT exhaustion	Intermittent outbound failures	All NAT ports used	NAT gateway metrics, conntrack
Firewall blocking	Connection timeout or refused	Security group/ACL blocking traffic	iptables, cloud security rules, firewall logs
Asymmetric routing	Connection works one way but not the other	Different request and response paths	Route tables on both sides, network design
Wrong next hop	Traffic sent to wrong gateway	Incorrect route or misconfiguration	ip route get, route table
Return traffic blocked	Request reaches destination but no response	Firewall or ACL blocking return path	Check inbound and outbound rules
SNAT/DNAT issue	Wrong source or destination IP	NAT rule misconfigured	NAT rules, conntrack, cloud NAT settings

5 CHECK NAT EXHAUSTION

```

# Check current NAT sessions
sudo conntrack -S

# View active NAT connections
sudo conntrack -L | grep src=<private_ip>

# Check ephemeral port range
cat /proc/sys/net/ipv4/ip_local_port_range

# Check count of connections per destination
ss -ant | awk '{print $5}' | sort | uniq -c | sort -nr
  
```

Signs of NAT exhaustion:

- New outbound connections fail.
- Intermittent internet access.
- Many connections in TIME_WAIT.
- All ephemeral ports used.

6 CHECK FIREWALL RULES

iptables (Linux)

```

# List current rules
sudo iptables -L -n -v

# Check specific port
sudo iptables -L -n | grep :80

# Check packet drops (if using ufw)
sudo ufw status verbose
  
```

What to look for:

- Rules blocking the required port or protocol.
- Incorrect source/destination IP or CIDR.
- Missing return traffic rule.
- Cloud security group or network ACL misconfiguration.
- Packet drops in firewall logs.

7 TROUBLESHOOTING FLOW

- 1 Check basic connectivity (ping, nc, traceroute).
- 2 Verify local routing table.
- 3 Confirm correct next hop and gateway.
- 4 Check firewall/ACL/security group rules.
- 5 Verify NAT configuration and available ports.
- 6 Look for asymmetric routing.
- 7 Check connection tracking and logs.
- 8 Test from both source and destination.

8 COMMON CAUSES

- Missing or incorrect route table entries.
- Firewall or security group blocking traffic.
- NAT port exhaustion.
- Asymmetric routing.
- Incorrect NAT rule (SNAT/DNAT).
- Wrong gateway or next hop.
- Network ACL denying traffic.
- Return traffic blocked.
- Overlapping CIDR or IP conflict.
- Misconfigured cloud network settings.

9 BEST PRACTICES

- Use proper route table design and least privilege firewall rules.
- Monitor NAT gateway/instance metrics.
- Set appropriate ephemeral port range.
- Enable logging for firewall and NAT.
- Validate both inbound and outbound rules.
- Avoid asymmetric routing.
- Use connection tracking and monitoring.
- Document network architecture and rules.

★ SENIOR ENGINEER NOTE

Routing, NAT and firewall issues can cause complete service outages. Always verify the full network path, check both directions of traffic, and correlate with logs, metrics and infrastructure configuration.

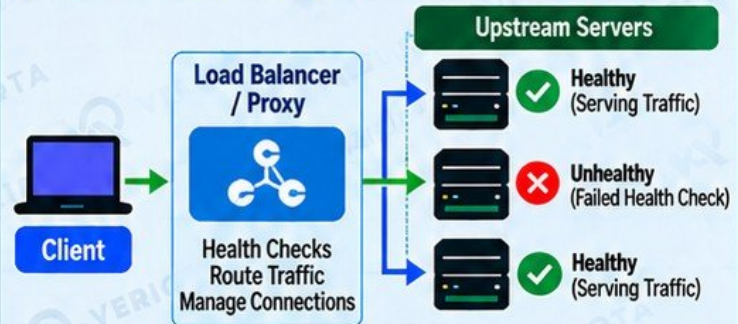
Load Balancer & Proxy Failures

Health checks, 502/503/504, upstream failures, connection reuse

1 KEY CONCEPTS

- A **load balancer** distributes traffic across multiple backend servers.
- **Health checks** determine if backend servers are healthy.
- **Proxies** sit between clients and backend services (e.g. Nginx, HAProxy, cloud load balancers).
- **502 Bad Gateway** means the proxy received an invalid response from the upstream server.
- **503 Service Unavailable** means no healthy upstream servers are available.
- **504 Gateway Timeout** means the proxy did not receive a timely response from the upstream server.
- **Connection reuse** keeps TCP connections open for multiple requests to improve performance.

2 HOW IT WORKS



3 USEFUL COMMANDS (LINUX)

```
# Test load balancer endpoint
curl -v http://your-lb-domain.com

# Check response time and status code
curl -o /dev/null -s -w "HTTP: %{http_code} Time: %{time_total}s" http://your-lb-domain.com

# Test multiple requests (connection reuse)
for i in {1..5}; do curl -s -o /dev/null -w "%{http_code}\n" http://your-lb-domain.com; done

# Check backend server health (directly)
curl -v http://backend-server:port/health

# View active connections to backend
ss -antp | grep :80

# Check proxy or load balancer logs (Nginx example)
sudo tail -f /var/log/nginx/access.log /var/log/nginx/error.log
```

4 COMMON HTTP ERRORS

Error Code	Meaning	Common Cause	What to Check
502 Bad Gateway	Invalid response from upstream server	Backend server crash, misconfiguration, incompatible response	Check backend server health, logs, and response.
503 Service Unavailable	No healthy upstream servers available	All backend servers failing health checks or overloaded	Check health checks, backend server status, and scaling.
504 Gateway Timeout	Upstream server did not respond in time	Slow backend, network latency, or timeout misconfiguration	Check backend latency, timeout settings, and logs.

5 CHECK HEALTH CHECKS

```
# Check health endpoint manually
curl -v http://backend-server:port/health

# Check health check path from load balancer (if available)
curl -v http://your-lb-domain.com/health

# Verify backend server is listening
ss -lntp | grep :<port>
```

Common health check issues:

- Wrong health check path.
- Backend returns non 200 status code.
- Slow response time.
- Health check interval too short.
- Firewall blocking health checks.
- Application not fully started.

6 CHECK CONNECTION REUSE

```
# View established connections
ss -antp | grep :80 | awk '{print $1}' | sort | uniq -c

# Check keep-alive settings (Nginx example)
cat /etc/nginx/nginx.conf | grep keepalive

# Monitor connection count over time
watch -n 1 'ss -ant | grep ESTAB | wc -l'
```

Why connection reuse matters:

- Reduces latency.
- Improves performance.
- Prevents too many new connections.
- Misconfiguration can lead to connection exhaustion.

7 COMMON CAUSES

- Unhealthy backend servers.
- Incorrect health check configuration.
- Backend server overload (high CPU, memory, or connection limits).
- Network latency between load balancer and backend.
- Firewall or security group blocking traffic.
- Misconfigured timeout settings.
- Application errors or slow responses.
- Connection pool exhaustion.
- DNS issues for backend servers.

8 TROUBLESHOOTING FLOW

- 1 Check the error code (502, 503, 504).
- 2 Verify load balancer and proxy health.
- 3 Check backend server health and logs.
- 4 Test backend connectivity directly.
- 5 Review health check configuration.
- 6 Check network, firewall and security rules.
- 7 Validate timeout and connection settings.
- 8 Monitor connection count and server resources.

9 BEST PRACTICES

- Use multiple healthy backend servers.
- Configure proper health check endpoints.
- Set appropriate timeout values.
- Enable connection reuse and keep-alive.
- Monitor backend server metrics.
- Implement auto scaling for high traffic.
- Use meaningful health check paths.
- Log and monitor load balancer and proxy errors.
- Regularly test failover and recovery.

10 MONITORING METRICS

- Health check success and failure rate.
- HTTP 502/503/504 error rates.
- Backend server response time (latency).
- Active and idle connection count.
- Connection reuse rate.
- Request rate (RPS).
- Backend server CPU and memory usage.
- Load balancer and proxy logs.
- Upstream server availability.

★ SENIOR ENGINEER NOTE

Load balancer and proxy failures can cause complete outages. Always monitor health checks, backend performance, connection counts and error rates, and configure timeouts and retries correctly.

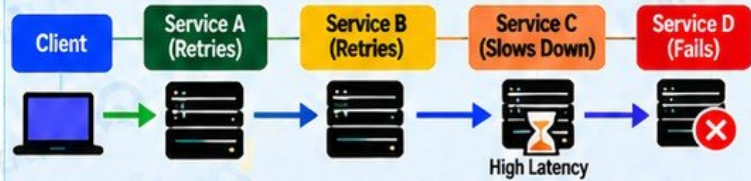
Timeouts, Retries & Cascading Failure

Retry amplification, backoff, jitter, timeout budgets, circuit breaking

1 KEY CONCEPTS

- **Timeouts** stop requests from waiting indefinitely.
- **Retries** help handle transient failures but can make things worse if not controlled.
- **Retry amplification** happens when multiple layers retry the same request, increasing load.
- **Backoff** increases the wait time between retries.
- **Jitter** adds randomness to avoid retry storms.
- **Timeout budgets** allocate time across multiple dependent services.
- **Circuit breaking** stops requests to a failing service to prevent cascading failures.
- **Cascading failure** happens when a failure in one service spreads to many other services.

2 HOW CASCADING FAILURE HAPPENS



- 1 A small issue causes high latency in Service C.
- 2 Service B retries multiple times.
- 3 Service A also retries.
- 4 Traffic increases rapidly (retry amplification).
- 5 Other services become overloaded and start failing.
- 6 This leads to a cascading failure across the system.

3 USEFUL COMMANDS (LINUX)

```
# Test request with timeout
curl --max-time 5 -v https://example.com

# Test retries (simulate)
for i in {1..5}; do curl -s -o /dev/null -w "%{http_code}\n" https://example.com; sleep 1; done

# Check response time
curl -o /dev/null -s -w "Time: %{time_total}s\n" https://example.com

# Simulate slow endpoint
nc -l -p 8080 # then delay response

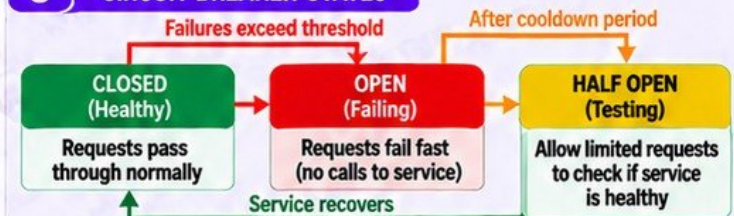
# Check open connections (possible retry storm)
ss -antp | grep :8080 | wc -l

# View application logs for timeouts
sudo tail -f /var/log/app.log | grep -iE "timeout|retry|circuit"
```

4 RETRY STRATEGIES

Strategy	How it Works	Example Use Case	Pros	Cons
No Retry	Send request once	Non idempotent operations	Simple, no extra load	Fails on transient errors
Fixed Delay	Retry after fixed time (e.g. 1s)	Simple services	Easy to implement	Can cause retry storms
Exponential Backoff	Increase wait time (1s, 2s, 4s, 8s)	Transient network failures	Reduces load over time	Slower recovery
Exponential Backoff with Jitter	Add randomness to wait time	Distributed systems	Prevents many clients retrying at the same time	Slightly more complex

5 CIRCUIT BREAKER STATES



6 TIMEOUT BUDGETS

- Set an overall timeout for the entire request (e.g. 3s).
- Divide the budget across all dependent services.
- Include time for retries and backoff.
- Account for network latency and processing time.

Example (3s total budget):

- Service A: 1s
- Service B: 1s
- Service C: 1s (including retries)



7 COMMON CAUSES

- Slow or overloaded service.
- Incorrect timeout settings.
- Aggressive retry policies.
- No jitter, causing synchronized retries.
- Lack of circuit breaking.
- Dependency failures (database, external API).
- Network latency or packet loss.
- Resource exhaustion (CPU, memory, connection limits).

8 SIGNS OF CASCADING FAILURE

- Increasing response times.
- High retry count in logs and metrics.
- Growing number of open connections.
- Multiple services reporting 5xx errors.
- CPU and memory usage increasing.
- Thread or connection pool exhaustion.
- Queue lengths increasing.
- Downstream services becoming unavailable.

9 TROUBLESHOOTING FLOW

- 1 Identify the failing service and error type.
- 2 Check response times and latency.
- 3 Review retry configuration and logs.
- 4 Look for retry amplification across services.
- 5 Check timeout and circuit breaker settings.
- 6 Analyze metrics (CPU, memory, connections, queue length).
- 7 Verify network health and external dependencies.
- 8 Implement fixes and monitor recovery.

10 BEST PRACTICES

- Set appropriate timeouts for each service.
- Use exponential backoff with jitter.
- Avoid unlimited retries.
- Implement circuit breakers.
- Set and enforce timeout budgets.
- Monitor retry counts, latency and error rates.
- Use idempotent requests where possible.
- Set proper connection and thread pool limits.
- Test failure scenarios (chaos testing).

★ REAL WORLD EXAMPLE

- An external API becomes slow.
- Service B retries 3 times, Service A retries 3 times, and 100 clients are making requests. This can turn 100 requests into 900 requests, overloading the downstream service and causing a cascading failure across the system.

★ SENIOR ENGINEER NOTE

Timeouts, retries and circuit breaking are critical for building resilient systems. Always design for failure, set sensible limits, and monitor for early signs of cascading failures.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Docker Container Won't Start

Entrypoint, environment, mounts, permissions, runtime evidence

COMMON SYMPTOMS

- Container exits immediately
- Container in Restarting state
- Error in docker logs
- Exit code not zero
- Application not listening on expected port
- Permission denied errors
- File or directory not found

DOCKER STARTUP FLOW



A failure at any stage can stop the container from starting.

1 CHECK CONTAINER STATUS

```

docker ps -a
docker inspect <container_id>
docker logs <container_id>
docker logs --tail 100 <container_id>
  
```

Look for:

- Status (exited, restarting)
- Exit code
- Error message
- Started and finished time

EXAMPLE OUTPUT (docker ps -a)

CONTAINER ID	IMAGE	STATUS
a1b2c3d4e5f6	myapp:latest	Exited (1) 5s ago

2 CHECK LOGS AND EXIT CODE

```

docker logs <container_id>
docker logs --tail 100 <container_id>
docker inspect --format='{{.State.ExitCode}}' \
  <container_id>
  
```

Common exit codes:

- 0 Successful exit
- 1 General error
- 126 Permission problem
- 127 Command not found
- 137 Killed (OOM or SIGKILL)

EXAMPLE (docker logs)

```

/docker-entrypoint.sh: line 8: \
/app/start.sh: Permission denied
or
exec: "nginx": executable file
not found in $PATH
  
```

3 VERIFY ENTRYPPOINT AND CMD

```

docker inspect --format='{{.Config.Entrypoint}}' \
  <image>
docker inspect --format='{{.Config.Cmd}}' <image>
  
```

Check if:

- Entrypoint path exists
- Command is correct
- Script has execute permission
- CMD/Entrypoint is overridden

4 CHECK ENVIRONMENT VARIABLES

```

docker inspect --format='{{.Config.Env}}' \
  <container_id>
docker run --rm <image> env
  
```

- Check required variables are set
- Look for missing or incorrect values

5 CHECK VOLUMES AND MOUNTS

```

docker inspect <container_id> --format='{{.Mounts}}'
docker run --rm -v /host/path:/app <image> ls -l /app
docker exec -it <container_id> ls -l <path>
  
```

- Ensure host path exists
- Check correct mount path in container
- Verify read/write permissions
- Check for missing config files or secrets

6 CHECK PERMISSIONS

```

docker exec -it <container_id> id
docker exec -it <container_id> ls -l <path>
docker exec -it <container_id> whoami
  
```

- Verify container user (root or non-root)
- Check file and directory permissions
- Ensure entrypoint script is executable
- Check ownership of mounted volumes

COMMON CAUSES AND FIXES

Entrypoint not found	→ Correct path or install missing package
Permission denied	→ Add execute permission or use correct user
Missing environment variable	→ Set required variables (docker run -e or compose)
Volume mount issue	→ Ensure host path exists and correct permissions
Command not found	→ Use full path or install the binary
Application exits immediately	→ Check logs and fix application configuration
Port not listening	→ Verify application config and exposed port

PRODUCTION TROUBLESHOOTING CHECKLIST

- ✓ Confirm container status and exit code
- ✓ Review logs for the actual error
- ✓ Verify entrypoint and command
- ✓ Check required environment variables
- ✓ Validate volume mounts and file permissions
- ✓ Ensure required files and dependencies exist
- ✓ Run the container interactively for debugging
- ✓ Fix the root cause and test again

★ SENIOR ENGINEER TIP

When debugging, run the container with an interactive shell to explore the filesystem and environment:
`docker run -it --entrypoint /bin/sh <image>`



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

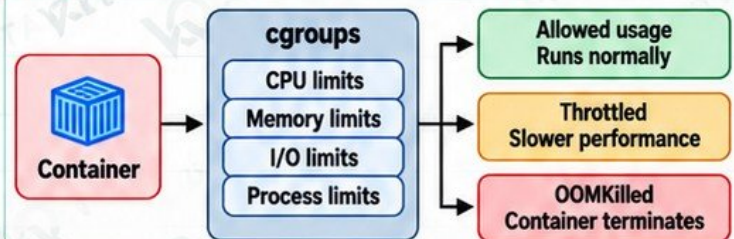
Container CPU & Memory Failures

cgroups, limits, throttling, OOM, noisy-neighbor behavior

1 COMMON SYMPTOMS

- Application is slow or unresponsive
- High CPU usage but low throughput
- Container CPU throttling
- OOMKilled (exit code 137)
- Container restarted repeatedly
- Node is healthy but one container is noisy
- Performance impact on other containers

2 CONTAINER RESOURCE FLOW



3 CHECK CPU USAGE

```

docker stats
docker inspect <container_id> --format '{{.HostConfig}}'
docker exec -it <container_id> top
docker exec -it <container_id> cat /sys/fs/cgroup/cpu.max
docker exec -it <container_id> cat /sys/fs/cgroup/cpu.stat
  
```

Look for:

- High CPU usage
- High throttling (nr_throttled, throttled_time)
- Load average inside container
- CPU limit too low for the workload

4 CHECK MEMORY USAGE

```

docker stats
docker exec -it <container_id> cat /sys/fs/cgroup/memory.max
docker exec -it <container_id> cat /sys/fs/cgroup/memory.current
docker exec -it <container_id> cat /proc/meminfo
docker inspect <container_id> --format '{{.HostConfig.Memory}}'
  
```

Look for:

- Memory usage close to limit
- oom_events increasing
- High page cache or application memory growth
- Swap usage (if enabled)

5 UNDERSTAND CPU THROTTLING

- Containers share CPU using Linux cgroups.
- If CPU limit is set, the Linux scheduler can throttle the container when it exceeds the limit.
- Throttling causes higher latency and slower processing even if the node has free CPU.

Check throttling metrics:

```
docker exec -it <container_id> cat /sys/fs/cgroup/cpu.stat
```

```

nr_periods      123456
nr_throttled    1234
throttled_time  567890000000
  
```

6 UNDERSTAND OOMKILL

- When a container exceeds its memory limit, the kernel terminates it (OOMKilled).
- Exit code is 137.
- You will see OOM events in container logs and cgroup memory events.
- An OOM kill can be caused by memory leak or a sudden memory spike.

Check OOM events:

```
docker exec -it <container_id> cat /sys/fs/cgroup/memory.events
```

```

low      0
high     0
max      5
oom      5
oom_kill 5
  
```

7 NOISY-NEIGHBOR BEHAVIOR

- A container without proper limits can consume most node resources.
- Other containers on the same node experience slow responses and higher latency.
- This is common when CPU or memory limits are not set.

8 BEST PRACTICES

- Set appropriate CPU and memory requests and limits.
- Monitor CPU throttling and memory usage.
- Use HPA or VPA where suitable.
- Identify and isolate noisy containers.
- Investigate memory leaks and right-size the application.
- Set alerts for high throttling and OOM events.



SENIOR ENGINEER TIP

Always set resource requests and limits. Monitor throttling and OOM events. Right-size based on real production metrics, not guesswork.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

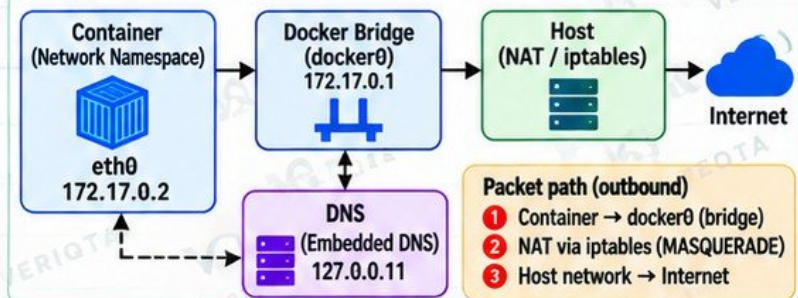
Container Networking Failures

Namespaces, bridges, DNS, port mapping, packet path

1 COMMON SYMPTOMS

- Container cannot reach external services
- Cannot reach another container
- DNS resolution failures
- Port mapping not working
- Connection timeouts or refused
- Works on host but not inside container
- Intermittent connectivity

2 CONTAINER NETWORKING OVERVIEW



3 CHECK CONTAINER NETWORK

```

docker inspect <container_id> --format '{{.NetworkSettings}}'
docker exec -it <container_id> ip a
docker exec -it <container_id> ip route
docker exec -it <container_id> ping <ip>
docker exec -it <container_id> curl -v <url>
docker exec -it <container_id> nslookup <domain>
  
```

4 CHECK DOCKER NETWORKS

```

docker network ls
docker network inspect bridge
docker network inspect <network_name>
docker network connect <network_name> <container_id>
docker network disconnect <network_name> <container_id>
  
```

5 DNS DEBUGGING

Containers use an embedded DNS server (127.0.0.11). It resolves container names and external domains.

```

docker exec -it <container_id> cat /etc/resolv.conf
docker exec -it <container_id> nslookup <domain>
docker exec -it <container_id> dig <domain>
docker exec -it <container_id> ping <service_name>
  
```

Common DNS issues:

- Cannot resolve service name
- No external DNS resolution
- Wrong DNS server configured
- Network policy blocking DNS
- Custom DNS misconfiguration

6 PORT MAPPING (HOST ↔ CONTAINER)

Check port mapping:

```

docker ps
docker port <container_id>
docker inspect <container_id> \
  --format '{{.NetworkSettings.Ports}}'
  
```

Common issues:

- Container port not exposed
- Wrong host port mapping
- Service listening only on localhost (127.0.0.1)
- Firewall blocking host port
- Application not listening on expected port

7 PACKET PATH INSIDE A CONTAINER



8 COMMON CAUSES AND FIXES

Cause	Fix / What to check
Container cannot reach internet	Check bridge, iptables NAT, host firewall, DNS
Cannot reach another container	Verify same network, service name, network policies
DNS not working	Check /etc/resolv.conf, 127.0.0.11, upstream DNS
Port not accessible from host	Check docker run -p, application bind address, firewall
Connection refused	Check if application is running and listening on correct port
Intermittent connectivity	Check resource limits, network policy, host networking issues

9 TROUBLESHOOTING CHECKLIST

- ✓ Confirm container network mode (bridge, host, custom)
- ✓ Check container IP and routes
- ✓ Verify DNS resolution
- ✓ Test connectivity to IP and domain
- ✓ Inspect port mapping and listening addresses
- ✓ Check docker network and firewall rules
- ✓ Validate network policies (if using Kubernetes)
- ✓ Check application logs and configuration
- ✓ Use packet capture if needed (tcpdump)

★ SENIOR ENGINEER TIP

Always isolate the failure domain. Test from inside the container, from the host, and from another container to identify where the connectivity breaks.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

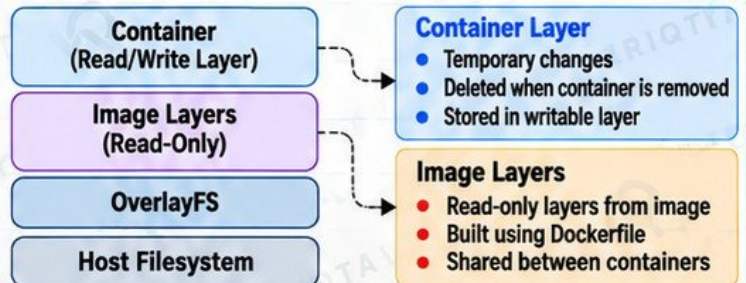
Container Storage & Image Failures

OverlayFS, volumes, disk growth, layers, registry/image-pull problems

1 COMMON SYMPTOMS

- Container fails to start due to storage errors
- Image pull failures from registry
- No space left on device
- Slow container startup
- Read/write permission denied
- Volume mount failures
- Large image size and long pull time
- Container exits due to missing files or layers

2 CONTAINER STORAGE OVERVIEW



3 CHECK DISK USAGE

```
docker system df
docker system df -v
docker images
docker container ls -a --size
df -h
du -sh /var/lib/docker
```

Look for:

- High disk usage on host
- Large images
- Many unused containers
- Build cache consuming space
- Logs growing inside containers

4 CHECK IMAGE DETAILS

```
docker images
docker image inspect <image>
docker history <image>
docker image ls --format '{{.Repository}}\n{{.Tag}}\t{{.Size}}'
```

5 VOLUME AND MOUNT TROUBLESHOOTING

```
docker inspect <container> --format '{{.Mounts}}'
docker volume ls
docker volume inspect <volume>
ls -lh <host_path>
docker run --rm -v <host_path>:/data alpine ls -lh /data
```

Common issues:

- Host path does not exist
- Wrong mount path in container
- Permission denied
- Read-only filesystem
- Volume not attached
- Data not persisting

6 IMAGE PULL FAILURES

```
docker pull <image>
docker login <registry>
docker pull <registry>/<image>:<tag>
docker system prune -a
docker image rm <image>
```

Common causes:

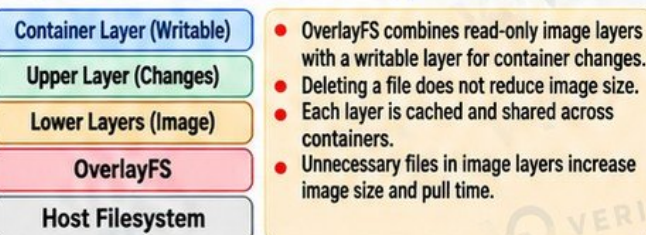
- Wrong image name or tag
- Authentication required
- Network or DNS issues
- Rate limiting (e.g. Docker Hub)
- Registry unreachable
- Corrupted or incomplete image

7 CONTAINER IMAGE BUILD FLOW



Large images and unnecessary layers increase build time, pull time and disk usage.

8 OVERLAYFS AND LAYERS



9 COMMON INCIDENTS AND FIXES

Incident	Fix / What to check
No space left on device	Clean up unused images, containers, volumes docker system prune -a
Image pull access denied	Verify image name, tag and registry login
Slow image pull	Check network, DNS, registry rate limits
Volume mount permission denied	Check host path permissions and user (UID/GID)
Container cannot find file	Verify mount path and Dockerfile COPY/ADD
Image exceeds size limit	Use multi-stage build and .dockerignore



SENIOR ENGINEER TIP

Keep images small, clean up unused resources, use .dockerignore, and monitor disk usage on the host. Always validate volumes and permissions before deployment.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

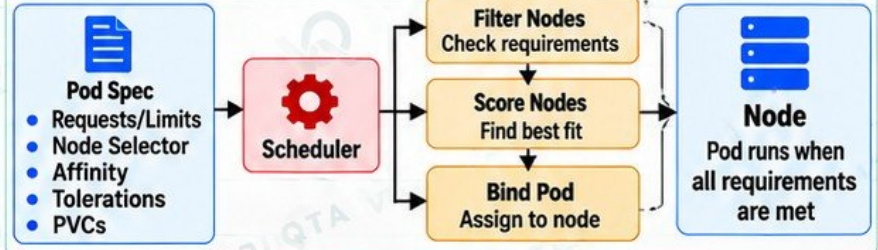
Kubernetes Pod Pending

Scheduler filters, requests, affinity, taints, PVCs, capacity

1 COMMON SYMPTOMS

- Pod stays in Pending state
- Pod is not scheduled to any node
- Events show scheduling errors
- Waiting for PVC to be bound
- Insufficient resources (CPU/Memory)
- Node affinity or taints not satisfied
- No available nodes or zone capacity
- Pod stuck for a long time

2 POD SCHEDULING FLOW



3 CHECK POD STATUS

```
kubectl get pods
kubectl get pod <pod-name> -o wide
kubectl describe pod <pod-name>
kubectl get events --sort-by=.lastTimestamp
kubectl get pods -n <namespace>
```

Look for:

- Pod status (Pending)
- Events and messages
- Node that was selected (if any)
- Reason for failure
- Related resources (PVC, etc)

4 COMMON EVENT MESSAGES

0/5 nodes are available:

- > insufficient cpu
- > insufficient memory
- > node(s) had taints that the pod did not tolerate
- > persistentvolumeclaim "xxx" is not bound
- > node selector mismatch
- > pod has unbound immediate PersistentVolumeClaims

5 CHECK RESOURCE REQUESTS

```
kubectl get nodes
kubectl describe node <node-name>
kubectl top nodes
kubectl describe pod <pod-name> |
grep -A5 "Requests:"
```

Check:

- CPU and memory requests
- Are requests too high?
- Available capacity on nodes
- Overcommit or quota limits
- Cluster autoscaler (if enabled)

6 CHECK NODE AFFINITY AND SELECTORS

```
kubectl describe pod <pod-name> |
grep -A5 Node-Selector
kubectl describe pod <pod-name> |
grep -A10 Affinity
kubectl get nodes --show-labels
```

Check:

- Node selector labels
- Node affinity rules
- Are matching nodes available?
- Topology (zone/region)

7 CHECK TAINTS AND TOLERATIONS

```
kubectl get nodes -o
custom-columns=NAME:.metadata.
name,TAINTS:.spec.taints
kubectl describe pod <pod-name> |
grep -A5 Tolerations
```

Check:

- Node taints
- Pod tolerations
- Do tolerations match?
- Are all nodes tainted?
- Control-plane node taints

8 CHECK PERSISTENT VOLUME CLAIMS

```
kubectl get pvc
kubectl describe pvc <pvc-name>
kubectl get storageclass
kubectl get pv
```

Check:

- PVC status (Pending)
- StorageClass exists
- Matching access modes
- Available capacity
- Volume binding mode

9 CHECK CLUSTER CAPACITY

```
kubectl get nodes
kubectl top nodes
kubectl describe node <node-name>
kubectl get resourcequota
--all-namespaces
```

Check:

- Sufficient nodes in the cluster
- CPU and memory usage
- Cluster autoscaler status
- Resource quotas and limits
- Zone or region capacity issues

10 EXAMPLE: EVENT OUTPUT

Events:				
Type	Reason	Age	From	Message
Warning	FailedScheduling	2m	default-scheduler	0/5 nodes are available: insufficient memory 1 node(s) had taints that the pod did not tolerate 2 node(s) didn't match node selector 1 pod has unbound immediate PVC

11 COMMON CAUSES AND FIXES

Cause	Fix / What to do
Insufficient CPU or memory	Reduce requests or add more nodes
Node selector mismatch	Correct labels or selector
Affinity rules not satisfied	Update affinity or add matching nodes
Taints not tolerated	Add tolerations or use different nodes
PVC not bound	Check StorageClass, capacity, access mode
No available nodes	Scale the cluster or enable Cluster Autoscaler
Quota limit reached	Increase quota or reduce resource requests

12 TROUBLESHOOTING CHECKLIST

- ✓ Confirm pod events and the exact reason
- ✓ Verify resource requests and node capacity
- ✓ Check node selectors and affinity rules
- ✓ Review node taints and tolerations
- ✓ Check PVC status and StorageClass
- ✓ Validate cluster autoscaler and node group size
- ✓ Ensure there are nodes in the correct zone/region
- ✓ Fix the issue and delete the pod to reschedule



SENIOR ENGINEER TIP

The scheduler must satisfy all requirements (resources, affinity, taints, PVCs, policies). Fix the most critical blocking reason shown in the events.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

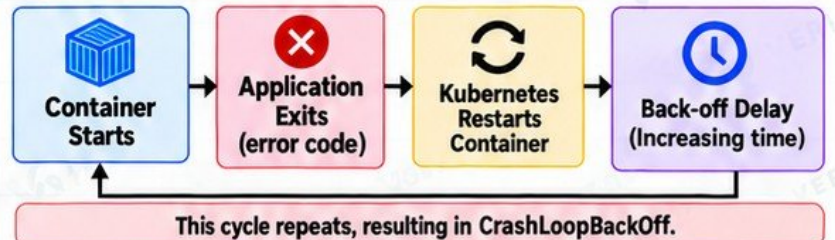
CrashLoopBackOff

Exit codes, previous logs, startup dependencies, probes, configuration

1 COMMON SYMPTOMS

- Pod status shows CrashLoopBackOff
- Container keeps restarting
- Application exits with an error code
- Startup fails due to missing dependency
- Readiness or liveness probe is failing
- Configuration or environment issue
- Works locally but fails in the cluster

2 POD LIFECYCLE (CRASHLOOPBACKOFF)



3 CHECK POD STATUS

```
kubectl get pods
kubectl describe pod <pod-name>
kubectl get pod <pod-name> -o wide
kubectl get events --sort-by=.lastTimestamp |
grep <pod-name>
```

Look for:

- Pod status (CrashLoopBackOff)
- Restart count
- Last termination reason
- Events and error messages
- Exit code
- Note the pod is running on

4 CHECK PREVIOUS LOGS

```
kubectl logs <pod-name>
kubectl logs <pod-name> --previous
kubectl logs <pod-name> -c <container>
kubectl logs <pod-name> -c <container>
--previous
```

Look for:

- Application error messages
- Stack traces
- Missing file or configuration
- Dependency connection failures
- Permission errors
- Environment variable issues
- Crash due to resource limits
- Startup script failures

5 CHECK EXIT CODE

```
kubectl get pod <pod-name> -o jsonpath=
'{}.status.containerStatuses[0].lastState.
terminated.exitCode'
```

Common exit codes:

- 0 Successful exit
- 1 General error
- 126 Permission problem
- 127 Command not found
- 137 Killed (OOM or SIGKILL)

6 CHECK STARTUP DEPENDENCIES

```
kubectl exec -it <pod-name> -- /bin/sh
# inside container
curl <dependency-service>
nslookup <service-name>
ping <service-name>
telnet <service-name> <port>
```

Check:

- Required services are running
- DNS resolution works
- Network connectivity
- Database or external service is reachable
- Secrets and configuration exist
- Correct environment variables

7 CHECK PROBES

```
kubectl describe pod <pod-name> |
grep -A5 "Liveness"
kubectl describe pod <pod-name> |
grep -A5 "Readiness"
kubectl get pod <pod-name> -o yaml
```

Common probe issues:

- Probe path or port is wrong
- Application not ready yet
- Probe timeout too short
- High startup time
- Application returns non 200
- Liveness probe killing the pod

8 CHECK CONFIGURATION AND ENVIRONMENT

```
kubectl describe pod <pod-name>
kubectl get configmap
kubectl get secret
kubectl env <pod-name>
kubectl exec -it <pod-name> -- printenv
```

Check for:

- Correct configuration files
- Required environment variables
- Secrets are mounted
- Correct command and arguments
- File permissions
- Valid application configuration

9 COMMON CAUSES AND FIXES

Cause	Fix / What to do
Application exits with error	Check logs, fix application error
Missing dependency	Ensure required service is running and reachable
Wrong configuration	Verify config files and environment variables
Permission denied	Fix file permissions or run as correct user
Liveness probe too aggressive	Increase initialDelay, timeout or fix probe path
OOMKilled (exit code 137)	Increase memory limit or reduce memory usage
Command not found (exit 127)	Fix command or install missing package
Crash due to invalid args	Check container command and arguments

10 TROUBLESHOOTING CHECKLIST

- Confirm pod status and restart count
- Check previous logs and exit code
- Identify the failure reason in events
- Verify startup dependencies and network connectivity
- Check configuration, secrets and environment variables
- Review and adjust probes if needed
- Check resource limits and node resources
- Fix the root cause and redeploy

11 EXAMPLE: DESCRIBE POD OUTPUT (EVENTS)

Type	Reason	Age	From	Message
Normal	Scheduled	2m	default-scheduler	Successfully assigned app-xyz to node-1
Normal	Pulling	2m	kubelet	Pulling image "myapp:latest"
Normal	Pulled	1m	kubelet	Successfully pulled image "myapp:latest"
Normal	Created	1m	kubelet	Created container app
Normal	Started	1m	kubelet	Started container app
Warning	BackOff	30s	kubelet	Back-off restarting failed container app

★ SENIOR ENGINEER TIP

Always check previous logs, events and exit codes. Most CrashLoopBackOff issues are caused by application errors, missing dependencies, misconfigured probes or incorrect configuration.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

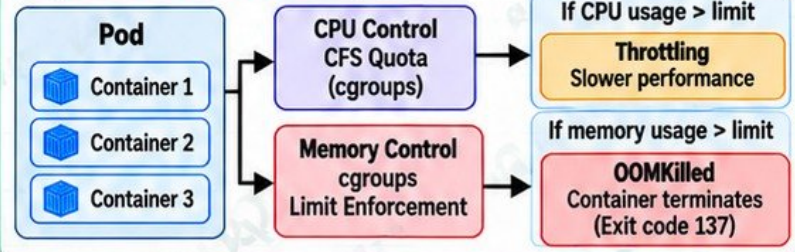
OOMKilled & CPU Throttling

Requests/limits, QoS, cgroups, working sets, right-sizing

1 COMMON SYMPTOMS

- Pod gets OOMKilled and restarts
- Container exits with code 137
- High memory usage before termination
- Container CPU throttling and poor performance
- High CPU usage but low actual throughput
- Application slow or unresponsive
- Pod remains in CrashLoopBackOff

2 HOW IT WORKS



3 CHECK POD STATUS

```
kubectl get pods
kubectl describe pod <pod-name>
kubectl get pod <pod-name> -o wide
kubectl get events --sort-by=.lastTimestamp | grep <pod-name>
```

Look for:

- Status: OOMKilled
- Last State: Terminated
- Exit code: 137
- Restart count increasing
- Events showing OOMKilled or CPU throttling
- Node where it happened

4 CHECK CONTAINER LOGS

```
kubectl logs <pod-name>
kubectl logs <pod-name> --previous
kubectl logs <pod-name> -c <container>
kubectl describe pod <pod-name> | grep -A5 "Last State"
```

Look for:

- Out of memory messages
- Slow processing or timeout errors
- Application memory errors
- Signs of CPU throttling in logs

5 CHECK RESOURCE USAGE

```
kubectl top pod <pod-name>
kubectl top pod <pod-name> --containers
kubectl top nodes
kubectl describe node <node-name> | grep -i allocatable
```

Look for:

- Memory usage close to limit
- High CPU usage with low throughput
- Node resource pressure
- Overall cluster resource usage

6 CHECK CONTAINER METRICS (cgroups)

```
kubectl exec -it <pod-name> -- cat /sys/fs/cgroup/memory.max
kubectl exec -it <pod-name> -- cat /sys/fs/cgroup/memory.current
kubectl exec -it <pod-name> -- cat /sys/fs/cgroup/memory.events
kubectl exec -it <pod-name> -- cat /sys/fs/cgroup/cpu.max
kubectl exec -it <pod-name> -- cat /sys/fs/cgroup/cpu.stat
```

7 UNDERSTAND KUBERNETES QoS

Guaranteed

requests = limits
for CPU and memory

- Most stable
- Less likely to be evicted

Burstable

requests < limits

- Can use extra resources if available
- May be evicted under pressure

BestEffort

No requests, no limits

- Highest risk of eviction
- Most likely to be OOMKilled

8 CPU THROTTLING ANALYSIS

```
kubectl exec -it <pod-name> -- cat /sys/fs/cgroup/cpu.stat
nr_periods 1234567
nr_throttled 45678
throttled_time 987654321
```

Look for:

- High nr_throttled and throttled_time
- CPU usage hitting the limit frequently
- Application latency and slower performance

9 COMMON CAUSES

OOMKilled

- Memory limit too low
- Memory leak in application
- Large in-memory data or cache
- Sudden memory spike
- Multiple containers sharing node memory
- Wrong memory requests/limits
- Node memory pressure

CPU Throttling

- CPU limit set too low
- Application requires more CPU
- High traffic or CPU intensive workload
- Noisy neighbor on the same node
- CPU limit lower than request in some cases
- Inefficient application code

10 FIXES AND BEST PRACTICES

- Set appropriate requests and limits based on real usage
- Monitor memory and CPU usage over time
- Use vertical or horizontal scaling as needed
- Optimize application memory and CPU usage
- Check for memory leaks and fix them
- Use proper QoS class for critical workloads (Guaranteed)
- Review node capacity and avoid noisy neighbors
- Use monitoring and alerts for OOM and throttling events

11 EXAMPLE: OOMKilled EVENT OUTPUT

Type	Reason	Age	From	Message
Warning	OOMKilled	2m	kubelet	Container killed due to OOM
Normal	Killing	2m	kubelet	Stopping container app
Normal	Pulled	2m	kubelet	Successfully pulled image
Normal	Created	2m	kubelet	Created container app
Normal	Started	2m	kubelet	Started container app

12 EXAMPLE: CPU THROTTLING METRICS

CONTAINER	CPU USAGE	THROTTLED %	THROTTLED TIME
app	950m	35%	320ms
worker	800m	25%	210ms
sidecar	100m	2%	10ms



SENIOR ENGINEER TIP

Right-size based on real production metrics. Set requests for stability and limits for protection. Monitor working sets, CPU throttling and OOM events, and adjust as usage patterns change.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

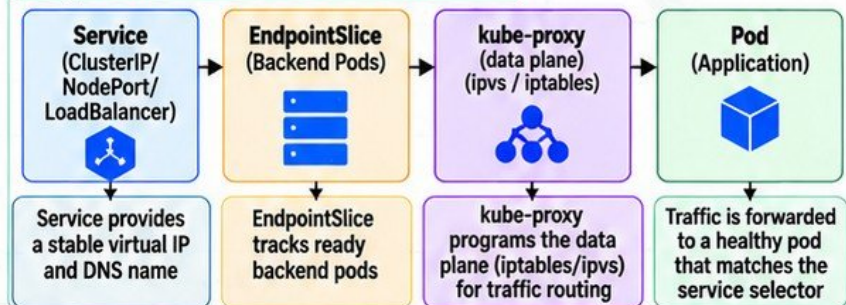
Kubernetes Service Connectivity

Service → EndpointSlice → kube-proxy/data plane → Pod

1 COMMON SYMPTOMS

- Cannot connect to the service
- Service is reachable but no response
- Connection refused
- Timeout while connecting
- Only some pods are reachable
- Works from inside the cluster but not outside
- Intermittent connectivity
- Service has no endpoints

2 SERVICE CONNECTIVITY FLOW



3 CHECK SERVICE

```
kubectl get svc
kubectl describe svc <service-name>
kubectl get svc <service-name> -o yaml
```

Check:

- Service type (ClusterIP/NodePort/LoadBalancer)
- Correct port and targetPort
- Service selector labels
- ClusterIP assigned
- External endpoints (if LoadBalancer)

4 CHECK ENDPOINTS LICE

```
kubectl get endpointslice
kubectl get endpointslice -l
kubernetes.io/service-name=<service-name>
kubectl describe endpointslice <name>
```

Check:

- Endpoints exist and match the service
- Endpoints are Ready (not terminating)
- Correct pod IPs and ports
- Label selector matches the pods

5 CHECK PODS AND LABELS

```
kubectl get pods -l <service-selector>
kubectl get pods -o wide
kubectl describe pod <pod-name>
```

Check:

- Pods are Running and Ready
- Labels match the service selector
- Container is listening on the target port
- No failing readiness probes
- Pod not terminating or crash looping

6 CHECK kube-proxy AND DATA PLANE

```
kubectl get pods -n kube-system -l k8s-app=kube-proxy
kubectl logs -n kube-system -l k8s-app=kube-proxy
iptables -t nat -L | grep <service-cluster-ip>
ipvsadm -ln | grep <service-cluster-ip>
```

Check:

- kube-proxy pods are running and healthy
- Correct iptables or IPVS rules exist
- Rules point to the correct pod endpoints

7 CONNECTIVITY TESTS

From inside the cluster
(test from a pod)

```
kubectl run -it --rm test --image=busybox -- sh
nslookup <service-name>
wget -qO- <service-name>:<port>
curl -v <service-name>:<port>
```

From a node

```
curl -v <cluster-ip>:<port>
curl -v <pod-ip>:<port>
nc -zv <cluster-ip> <port>
```

From outside (if applicable)

```
curl -v <node-ip>:<nodeport>
curl -v <loadbalancer-ip>:<port>
Check security groups and firewall
```

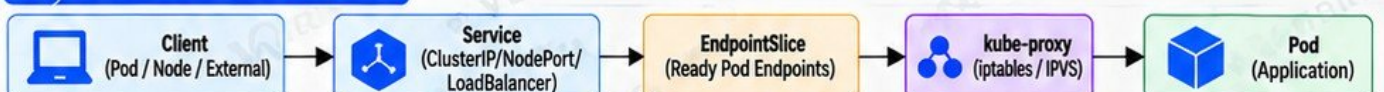
8 COMMON CAUSES

Cause	Details
No endpoints	Selector mismatch, pods not ready, all pods terminating
Wrong port	Incorrect port or targetPort
Pods not ready	Readiness probe failing, application not listening
Network policy	Traffic blocked by NetworkPolicy
kube-proxy issue	kube-proxy not running or incorrect rules
DNS issue	Service name not resolving
Firewall / security group	External access blocked (NodePort/LoadBalancer)
Pod not listening	Application listening on different port or interface

9 FIXES

- ✓ Ensure service selector matches pod labels
- ✓ Fix pod readiness probe issues
- ✓ Verify correct port and targetPort
- ✓ Ensure endpoints exist and are Ready
- ✓ Check and fix network policies
- ✓ Restart kube-proxy if needed
- ✓ Verify iptables or IPVS rules
- ✓ Check DNS resolution
- ✓ Validate firewall and security group rules for external access
- ✓ Confirm application is listening on the expected port

10 QUICK ARCHITECTURE VIEW



SENIOR ENGINEER TIP

Always verify each layer in order: Service → EndpointSlice → kube-proxy/data plane → Pod.
Use real connectivity tests from inside the cluster and from outside to narrow down the issue.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

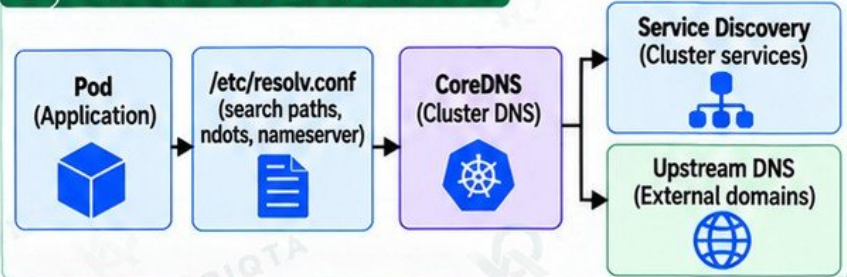
Kubernetes DNS Failures

CoreDNS, service discovery, search paths, upstream DNS, ndots

1 COMMON SYMPTOMS

- Cannot resolve service names
- DNS lookup times out
- Intermittent DNS failures
- Works for some services but not others
- Slow DNS resolution
- External domain resolution fails
- Application errors like "no such host"
- High latency when resolving names

2 HOW KUBERNETES DNS WORKS



3 CHECK DNS CONFIGURATION IN POD

```
kubectl exec -it <pod-name> -- cat /etc/resolv.conf
kubectl exec -it <pod-name> -- cat /etc/nsswitch.conf
kubectl exec -it <pod-name> -- grep ndots /etc/resolv.conf
```

Look for:

- Nameserver (usually CoreDNS service IP)
- Search paths (namespace.svc.cluster.local, etc)
- ndots value
- Options like timeout and attempts

4 TEST DNS RESOLUTION

```
kubectl exec -it <pod-name> -- nslookup <service-name>
kubectl exec -it <pod-name> -- nslookup <service>.<namespace>.svc.cluster.local
kubectl exec -it <pod-name> -- nslookup <service>.kubernetes.default.svc.cluster.local
kubectl exec -it <pod-name> -- nslookup external-domain.com
kubectl exec -it <pod-name> -- dig <service-name>
kubectl exec -it <pod-name> -- dig @<coredns-ip> <domain>
```

5 CHECK COREDNS STATUS

```
kubectl get pods -n kube-system -l k8s-app=kube-dns
kubectl describe pod <coredns-pod> -n kube-system
kubectl logs <coredns-pod> -n kube-system
kubectl get svc -n kube-system kube-dns
kubectl get configmap coredns -n kube-system
```

Look for:

- CoreDNS pods are Running
- No crash loops
- Errors in logs
- Correct configuration in ConfigMap
- Service IP and endpoints are correct

6 CHECK SERVICE DISCOVERY

```
kubectl get svc
kubectl get svc -n <namespace>
kubectl get endpoints <service-name>
kubectl get endpointslice -l <service-name>
kubectl describe svc <service-name>
kubectl describe endpointslice <name>
```

Look for:

- Service exists
- Correct namespace
- Endpoints are ready
- EndpointSlice has healthy pods
- Service selector matches pod labels

7 UNDERSTAND SEARCH PATHS AND NDOTS

Example /etc/resolv.conf

```
nameserver 10.96.0.10
search default.svc.cluster.local
svc.cluster.local
cluster.local
options ndots:5 timeout:2 attempts:3
```

How it works:

- If ndots value is high (default 5), short names are tried with search paths first.
- This can cause delays if the name is external.
- Names with fewer dots are treated as relative and appended with search paths.

Example resolution for "api" (with ndots:5)

- 1 api.default.svc.cluster.local
- 2 api.svc.cluster.local
- 3 api.cluster.local
- 4 api.cupstream-dns>
- 5 (Only after failures, try as external)

8 CHECK UPSTREAM DNS (EXTERNAL)

```
kubectl exec -it <coredns-pod> -- nslookup google.com
kubectl exec -it <coredns-pod> -- dig google.com
kubectl exec -it <coredns-pod> -- cat /etc/resolv.conf
```

Look for:

- CoreDNS can resolve external domains
- Correct upstream nameservers
- Firewall or network policy not blocking
- No timeouts or high latency

9 COMMON CAUSES

Cause	Details
CoreDNS not running	Pods crashed or not ready
Wrong service name	Typo or wrong namespace
No endpoints	Service exists but no ready pods
High ndots value	Causes delays for external domains
Network policy	DNS traffic blocked (UDP/TCP 53)
Upstream DNS issue	Cluster cannot reach external DNS
Misconfigured CoreDNS	Incorrect ConfigMap or plugin settings
Pod DNS config override	Custom dnsConfig causing wrong settings

10 FIXES AND BEST PRACTICES

- ✓ Ensure CoreDNS pods are healthy and properly configured
- ✓ Verify service names, namespaces and endpoints
- ✓ Use FQDN for external domains to avoid search path delays
- ✓ Adjust ndots if needed for external lookups
- ✓ Check network policies and firewall rules for DNS (UDP/TCP 53)
- ✓ Verify upstream DNS servers are reachable from CoreDNS
- ✓ Monitor CoreDNS logs and metrics
- ✓ Use shorter service names only for cluster services

11 REAL EXAMPLES

Internal service

```
kubectl exec -it pod -- nslookup my-service.my-namespace.svc.cluster.local
Server: 10.96.0.10
Address: 10.96.0.10#53
Name: my-service.my-namespace.svc.cluster.local
Address: 10.96.123.45
```

External domain

```
kubectl exec -it pod -- nslookup google.com
Server: 10.96.0.10
Address: 10.96.0.10#53
Name: google.com
Address: 142.250.190.78
```



SENIOR ENGINEER TIP

Use FQDN for external domains, monitor CoreDNS, and keep ndots at a reasonable value to avoid delays.

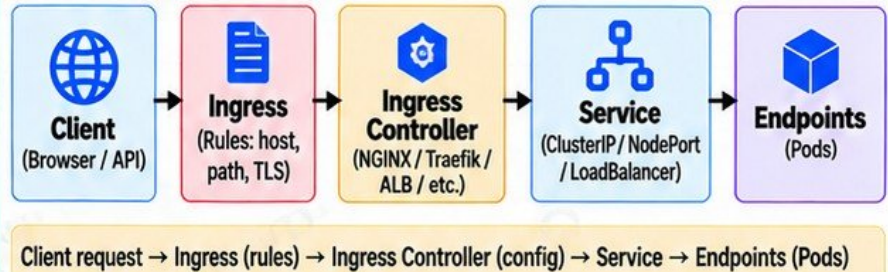
Kubernetes Ingress Failures

Ingress → controller → Service → endpoints; TLS and routing

1 COMMON SYMPTOMS

- Cannot access application via domain
- 504 Gateway Timeout
- 404 Not Found (wrong routing)
- TLS/SSL errors
- Ingress controller not processing rules
- Service or endpoints not reachable
- Works for some paths but not others
- Redirect loop or incorrect redirects
- Intermittent connectivity

2 INGRESS FLOW



3 CHECK INGRESS RESOURCE

```
kubectl get ingress
kubectl describe ingress <ingress-name>
kubectl get ingress <ingress-name> -o yaml
```

Look for:

- Correct host and path rules
- Ingress class matches controller
- TLS configuration and secret
- Annotations are correct
- Events and error messages

4 CHECK INGRESS CONTROLLER

```
kubectl get pods -n ingress-nginx
kubectl logs -n ingress-nginx <controller-pod>
kubectl describe pod <controller-pod>
kubectl get svc -n ingress-nginx
```

Look for:

- Controller is running and healthy
- No configuration errors in logs
- Service type and external IP/LoadBalancer
- Correct ingress class
- Events showing rejected or invalid configuration

5 CHECK SERVICE AND ENDPOINTS

```
kubectl get svc
kubectl get svc <service-name> -o wide
kubectl get endpoints <service-name>
kubectl get endpointslice
kubectl describe endpointslice <name>
```

Look for:

- Service exists and correct type
- Endpoints are ready (pods are healthy)
- Ports match targetPort
- EndpointSlice has correct pod IPs
- Selector matches pod labels

6 CHECK TLS/SSL

```
kubectl get secret
kubectl describe secret <tls-secret>
kubectl get ingress <ingress-name> -o yaml
```

Look for:

- TLS secret exists and is valid
- Certificate is not expired
- Correct hostname in certificate (CN/SAN)
- Ingress references the correct secret
- Controller can access the secret

7 TEST ROUTING

```
# Test DNS resolution
nslookup <domain>
dig <domain>
# Test HTTP/HTTPS
curl -I http://<domain>
curl -I https://<domain>
# Test specific path
curl -I https://<domain>/path
```

Look for:

- DNS resolves to ingress IP
- Correct HTTP status code
- Redirects working as expected
- Path based routing matches
- No 404 or 503 errors

8 COMMON CAUSES

Cause	Details
DNS issue	Domain not pointing to ingress LoadBalancer or external IP
Wrong ingress class	Controller not watching the ingress
TLS/SSL misconfiguration	Invalid or expired certificate
Service not found	Incorrect service name or namespace
No endpoints	Pods not ready or selector mismatch
Wrong port	Service port/targetPort mismatch
Incorrect path rules	Path or host rules misconfigured
Firewall / security group	Traffic blocked (cloud or on-prem)
Controller configuration	Invalid annotations or config

9 INGRESS CONTROLLER LOGS (EXAMPLE)

```
2026-10-01T10:15:23 [error] no endpoints available for service "app-service"
2026-10-01T10:16:10 [warn] backend returned 503
2026-10-01T10:17:05 [error] certificate "app-tls" not found
2026-10-01T10:18:12 [info] configured ingress for host "app.example.com"
```

10 EXAMPLE INGRESS (NGINX)

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: app-ingress
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
spec:
  ingressClassName: nginx
  tls:
  - hosts:
    - app.example.com
    secretName: app-tls
  rules:
  - host: app.example.com
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: app-service
            port:
              number: 80
```

11 FIXES AND BEST PRACTICES

- ✓ Ensure DNS points to the correct ingress IP
- ✓ Use the correct ingress class
- ✓ Verify TLS certificates and secrets
- ✓ Check that service and endpoints are healthy
- ✓ Validate host and path rules
- ✓ Use proper annotations for your controller
- ✓ Monitor ingress controller logs
- ✓ Implement correct redirects (HTTP to HTTPS)
- ✓ Use readiness probes for backend pods
- ✓ Test with curl and browser from outside
- ✓ Set up monitoring and alerts for ingress errors



SENIOR ENGINEER TIP

Always test each layer: DNS → Ingress → Controller → Service → Endpoints.
Most ingress issues are caused by DNS, wrong rules, missing endpoints or TLS misconfiguration.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

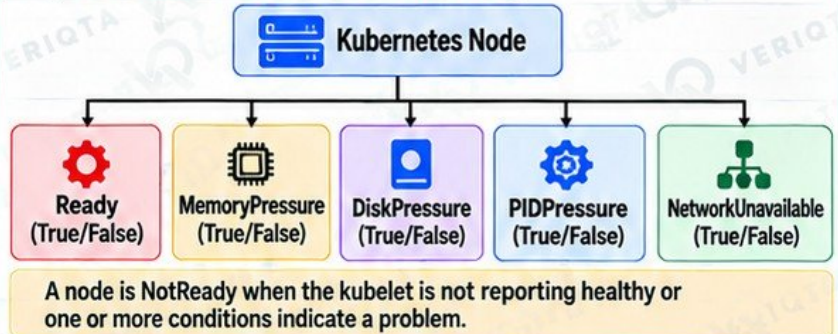
Node NotReady & Node Pressure

kubelet, conditions, memory/disk/PID pressure, eviction

1 COMMON SYMPTOMS

- Node status is NotReady
- Pods are evicted from the node
- Node shows memory, disk or PID pressure
- kubelet is not responding or crashed
- High resource usage on the node
- Pods stuck in Pending or rescheduled
- Frequent node flapping between Ready and NotReady
- Applications become unavailable

2 NODE STATUS AND CONDITIONS



3 CHECK NODE STATUS

```
kubectl get nodes -o wide
kubectl describe node <node-name>
kubectl get nodes
kubectl get events --sort-by=.lastTimestamp |
grep <node-name>
```

Look for:

- Node Ready status
- Condition states
- Recent events
- Taints on the node
- Node IP and hostname
- Age and kubelet version

4 CHECK NODE CONDITIONS

```
kubectl describe node <node-name> | grep -A 10
"Conditions:"
kubectl get node <node-name> -o json |
jq '.status.conditions'
kubectl get node <node-name> -o yaml
```

5 CHECK KUBELET STATUS (ON THE NODE)

```
sudo systemctl status kubelet
sudo journalctl -u kubelet -n 100
sudo journalctl -u kubelet -f
ps aux | grep kubelet
```

Look for:

- kubelet is running
- Errors in kubelet logs
- Authentication or network issues
- Node registration problems
- Frequent restarts

6 CHECK RESOURCE PRESSURE

```
# Memory
free -h
top
# Disk
df -h
# Inodes
df -i
# PID count
ps aux | wc -l
# cgroup info
cat /sys/fs/cgroup/memory/memory.usage_in_bytes
cat /sys/fs/cgroup/pids/pids.current
# Node resource view
kubectl top node <node-name>
```

7 UNDERSTAND NODE PRESSURE CONDITIONS

Condition	What it means	Common causes	Impact
Ready=False	Node is not healthy or kubelet not reporting	kubelet down, network issue, high load, system problem	Pods may be evicted or rescheduled
MemoryPressure	Node is low on memory	High memory usage, memory leak, insufficient memory	Pods may be evicted based on QoS
DiskPressure	Node is low on disk space or inodes	Large logs, images, emptyDir, container writable layer	Pods may be evicted, image pulls can fail
PIDPressure	Node has too many processes	Runaway processes, misbehaving containers	New pods may fail to start
NetworkUnavailable	Node network is not ready	CNI issue, network down	Pods on node lose network

8 CHECK EVICTION AND KUBELET CONFIG

```
kubectl describe node <node-name> | grep -A 20 "Events:"
sudo cat /var/lib/kubelet/config.yaml
sudo cat /var/lib/kubelet/kubelet-config.yaml
kubectl get pods --all-namespaces --field-selector
spec.nodeName=<node-name>
kubectl get events --field-selector involvedObject.kind=Node,
involvedObject.name=<node-name>
```

Look for:

- Eviction messages
- Eviction thresholds
- Imminent cgroup collection events
- kubelet configuration
- Failed image pulls due to disk pressure
- Repeated pod evictions

9 COMMON CAUSES

Cause	Details
Kubelet not running	Service stopped, crash, misconfiguration
High memory usage	Memory leak, large workloads, insufficient memory
High disk usage	Logs, container images, emptyDir, no log rotation
High inode usage	Many small files, logs
Too many processes	Runaway processes, misbehaving containers
Network issues	CNI plugin failure, network down
System resource limits	ulimit, cgroup limits, incorrect configuration

10 FIXES AND BEST PRACTICES

- Restart kubelet if it has crashed
- Free up memory and disk space
- Clean up unused container images and logs
- Set proper eviction thresholds
- Use resource requests and limits
- Monitor node metrics and alerts
- Use log rotation and monitoring
- Fix CNI or network issues
- Investigate and stop runaway processes
- Right size node instance type

11 EXAMPLE EVENT OUTPUT

Type	Reason	Age	From	Message
Warning	MemoryPressure	2m	kubelet	Node has condition: memory pressure
Warning	DiskPressure	5m	kubelet	Node has condition: disk pressure
Normal	NodeNotReady	6m	kubelet	Node kubelet is not ready
Warning	Evicted	7m	kubelet	The node was low on resource: memory. Container was evicted.



SENIOR ENGINEER TIP

Monitor node conditions, resource usage and kubelet logs. Set appropriate eviction thresholds and use requests and limits to prevent node pressure and unexpected pod evictions.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

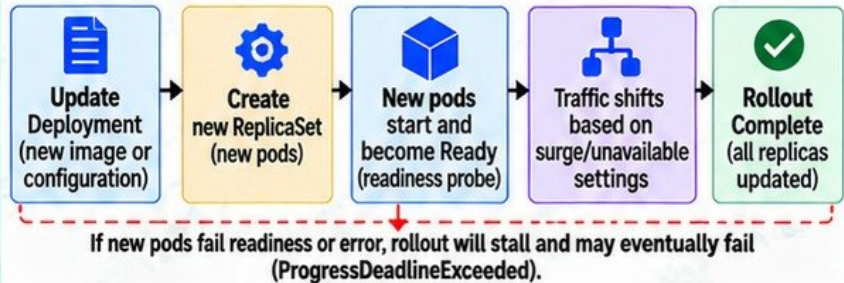
Failed Kubernetes Rollout

Deployment conditions, readiness, surge/unavailable, rollback decisions

1 COMMON SYMPTOMS

- Rollout gets stuck or does not complete
- New pods are not becoming Ready
- Deployment shows ProgressDeadlineExceeded
- High number of Unavailable replicas
- Readiness probe failures
- New version causes application errors
- Pods keep restarting during rollout
- Service experiencing downtime
- Need to rollback to previous version

2 ROLLOUT FLOW



3 CHECK DEPLOYMENT STATUS

```
kubectl get deployment <deploy-name>
kubectl describe deployment <deploy-name>
kubectl rollout status deployment <deploy-name>
kubectl get rs -l app=<app-name>
kubectl get pods -l app=<app-name>
```

Look for:

- Desired vs Available replicas
- Updated replicas count
- Deployment conditions
- Events and error messages
- ProgressDeadlineExceeded
- ReplicaSet status

4 CHECK DEPLOYMENT CONDITIONS

```
kubectl describe deployment <deploy-name> |
  grep -A 10 "Conditions:"
kubectl get deployment <deploy-name> -o json |
  jq ".status.conditions"
kubectl get deployment <deploy-name> -o yaml
```

Look for:

- Progressing = False (reason: ProgressDeadlineExceeded)
- Available = False (not enough ready replicas)
- ReplicaFailure conditions
- Clear reason and message for the failure

5 CHECK PODS AND READINESS

```
kubectl get pods -l app=<app-name> -o wide
kubectl describe pod <pod-name>
kubectl logs <pod-name>
kubectl get events --sort-by=.lastTimestamp |
  grep <pod-name>
```

Look for:

- Pods are running or crashing
- Readiness probe failures
- Application errors in logs
- Image pull or configuration errors
- Insufficient resources
- Events showing why pods are not becoming Ready

6 CHECK ROLLOUT HISTORY

```
kubectl rollout history deployment <deploy-name>
kubectl rollout history deployment <deploy-name> --revision=<n>
kubectl get rs -l app=<app-name>
```

Look for:

- Previous successful revision
- Compare changes between revisions
- Identify when the issue started

7 UNDERSTAND SURGE AND UNAVAILABLE

maxSurge

- Number of extra pods that can be created above the desired replicas during rollout
- Helps ensure zero downtime
- Can be a number or percentage

maxUnavailable

- Number of pods that can be unavailable during rollout
- Controls how many old pods can be taken down
- Can be a number or percentage
- Setting too low can slow rollout
- Setting too high can cause downtime

Example strategy

```
strategy:
  type: RollingUpdate
  rollingUpdate:
    maxSurge: 1
    maxUnavailable: 0
```

8 ROLLBACK OPTIONS

```
kubectl rollout undo deployment <deploy-name>
kubectl rollout undo deployment <deploy-name>
  --to-revision=<n>
kubectl get deployment <deploy-name>
```

Use rollback when:

- New version is causing failures
- Pods are not becoming Ready
- Application has critical issues
- Quickly restore the last known good version

9 COMMON CAUSES

Cause	Details
Readiness probe failure	Application not ready or wrong probe configuration
Image pull error	Wrong image name, tag or registry access issue
Configuration error	Invalid config, missing secrets or environment variables
Insufficient resources	Not enough CPU or memory on nodes
Progress deadline exceeded	Pods not becoming Ready within the timeout
CrashLoopBackOff	Application crashes during startup
Misconfigured surge/unavailable	Settings prevent rollout from completing
Dependency not available	Database, external service or DNS issue

10 FIXES AND BEST PRACTICES

- Check application logs and fix errors
- Validate readiness probe configuration
- Ensure image and registry access are correct
- Verify configuration, secrets and environment variables
- Increase resources if needed
- Check and adjust surge and unavailable settings
- Use rollout status to monitor progress
- Set a reasonable progress deadline
- Perform a controlled rollback if the new version is not stable
- Test the new version in a staging environment first

11 EXAMPLE OUTPUT (FAILED ROLLOUT)

```
$ kubectl rollout status deployment web-app
Waiting for deployment "web-app" rollout
to finish: 1 out of 3 new replicas
have been updated...
error: deployment "web-app"
exceeded its progress deadline

$ kubectl get deployment web-app
NAME      READY   UP-TO-DATE   AVAILABLE
web-app   1/3     3            1
```

★ SENIOR ENGINEER TIP

Always check deployment conditions, pod readiness and events first. Use surge and unavailable settings based on your application's tolerance for downtime, and rollback quickly when needed.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

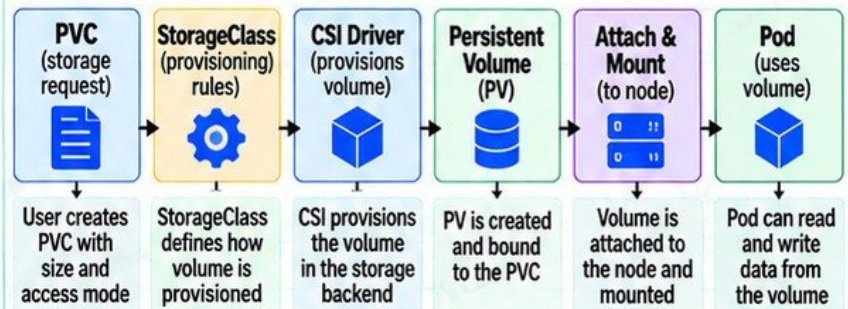
Persistent Volume Failures

PVC binding, StorageClass, CSI, attach/mount, topology

1 COMMON SYMPTOMS

- PVC stays in Pending state
- Pod stuck in ContainerCreating
- MountVolume or AttachVolume errors
- No matching StorageClass
- Insufficient storage capacity
- Node topology/zone mismatch
- Permission or filesystem errors
- Volume stuck in Terminating
- ReadWriteOnce volume attached to another node
- CSI driver or storage backend issues

2 PV, PVC AND STORAGE FLOW



3 CHECK PVC STATUS

```
kubectl get pvc
kubectl get pvc -n <namespace>
kubectl describe pvc <pvc-name>
kubectl get events --sort-by=.lastTimestamp
kubectl get pv
kubectl describe pv <pv-name>
```

Look for:

- PVC status (Pending / Bound)
- Events and error messages
- Requested storage size
- StorageClass name
- Access mode (RWO, RWX, ROX)
- Volume name (if bound)

4 CHECK STORAGECLASS

```
kubectl get storageclass
kubectl describe storageclass <name>
kubectl get storageclass -o yaml
```

Look for:

- Correct StorageClass name
- Provisioner (CSI driver)
- VolumeBindingMode (Immediate or WaitForFirstConsumer)
- ReclaimPolicy (Delete or Retain)
- Parameters (type, IOPS, replication, etc)

5 CHECK CSI DRIVER AND NODES

```
kubectl get csidrivers
kubectl get csinodes
kubectl describe csinodes
kubectl get nodes -o wide
kubectl describe node <node-name>
| grep -A 20 "Conditions"
```

Look for:

- CSI driver is installed and healthy
- CSI node objects exist
- Driver is registered on nodes
- Node has required conditions (Ready)
- Topology labels (zone, region)
- Taints that may block scheduling

6 CHECK POD MOUNT AND EVENTS

```
kubectl describe pod <pod-name>
kubectl get events --sort-by=.lastTimestamp | grep <pod-name>
kubectl logs <pod-name> -c <container-name> (if available)
```

Look for:

- MountVolume or AttachVolume errors
- Failed to find PVC or PV
- Permission denied or filesystem errors
- Volume already in use (ReadWriteOnce)
- Node affinity or topology mismatch
- Timeouts while attaching or mounting

7 UNDERSTAND ACCESS MODES

Access Mode	Description	Common Use Case
ReadWriteOnce (RWO)	Can be mounted by only one node	Typical for most applications
ReadOnlyMany (ROX)	Can be mounted read-only by many nodes	Shared read-only data
ReadWriteMany (RWX)	Can be mounted read-write by many nodes	Shared storage (NFS, Ceph, etc)

8 COMMON CAUSES

Cause	Details
No matching StorageClass	PVC cannot be provisioned
Insufficient storage capacity	Requested size exceeds available storage
Topology/zone mismatch	Volume in different zone than node
CSI driver issue	Driver not installed or misconfigured
Volume attachment limit	Node has reached max volumes
Wrong access mode	ReadWriteOnce used on multiple nodes
Permissions / filesystem	Mount fails due to permissions or wrong fs
Volume stuck in Terminating	Finalizers or backend issue
Network connectivity issue	Node cannot reach storage backend

9 TROUBLESHOOTING STEPS

- 1 Check PVC status and events
- 2 Verify StorageClass and provisioner
- 3 Confirm CSI driver and CSI node are healthy
- 4 Check node conditions, taints and topology
- 5 Review pod events and mount errors
- 6 Validate storage backend (cloud disk, NFS, etc)
- 7 Check access mode and volume attachment limits
- 8 Fix configuration and retry or recreate the PVC if needed

10 EXAMPLE ERRORS

```
Warning FailedScheduling 0/3 nodes are
available: 1 node(s) had volume node
affinity conflict.
MountVolume.SetUp failed for volume
"data": rpc error: code = Unavailable
desc = failed to attach volume.
persistentvolumeclaim "data" not found
Error: failed to mount /var/lib/kubelet/
pods/... permission denied
Multi-Attach error: volume is already
exclusively attached to one node
```

11 FIXES AND BEST PRACTICES

- ✓ Use the correct StorageClass and parameters
- ✓ Ensure sufficient storage capacity
- ✓ Match volume topology with node zone/region
- ✓ Verify CSI driver installation and health
- ✓ Use appropriate access mode for your workload
- ✓ Clean up and remove stuck volumes and finalizers
- ✓ Check and fix permissions or filesystem issues
- ✓ Monitor storage backend and node disk usage
- ✓ Use WaitForFirstConsumer for zonal storage
- ✓ Implement alerts for PVC Pending and mount failures

★ SENIOR ENGINEER TIP

Always check PVC events, StorageClass configuration, CSI driver health and node topology. Most persistent volume issues are caused by a mismatch between storage configuration and cluster state.

Kubernetes RBAC & Identity Failures

ServiceAccounts, authorization, workload identity, least privilege

1 COMMON SYMPTOMS

- Permission denied errors
- Pods cannot access the Kubernetes API
- ServiceAccount token issues
- Unauthorized when accessing resources
- RBAC rules not applied or incorrect
- Workload identity not configured
- Access works in one namespace but fails in another
- Applications fail to access cloud resources
- Overly permissive or missing permissions
- 403 Forbidden or 401 Unauthorized errors

2 KUBERNETES IDENTITY AND AUTHORIZATION FLOW



Request → Authenticate → Authorize (RBAC) → Allow or Deny → Access Resource

3 CHECK SERVICEACCOUNT

```

kubectl get sa -n <namespace>
kubectl describe sa <sa-name> -n <namespace>
kubectl get sa -A
kubectl get secret -n <namespace> \
| grep <sa-name>
  
```

Look for:

- ServiceAccount exists
- Correct namespace
- Automounted token
- Associated secrets
- Annotations (for workload identity)

4 CHECK RBAC RULES

```

kubectl get role,rolebinding -n <namespace>
kubectl get clusterrole,clusterrolebinding
kubectl describe role <role-name> -n <namespace>
kubectl describe rolebinding <name> -n <namespace>
kubectl auth can-i <verb> <resource> \
--as=system:serviceaccount:<namespace>:<sa-name>
  
```

5 TEST AUTHORIZATION

```

kubectl auth can-i get pods \
--as=system:serviceaccount:default:my-sa
kubectl auth can-i list deployments \
--as=system:serviceaccount:prod:app-sa
kubectl auth can-i --list \
--as=system:serviceaccount:prod:app-sa
  
```

Look for:

- Whether the action is allowed
- Correct verb, resource and namespace
- Compare with expected permissions
- Identify missing permissions

6 SERVICEACCOUNT EXAMPLE (YAML)

```

apiVersion: v1
kind: ServiceAccount
metadata:
  name: app-sa
  namespace: prod
  annotations:
    eks.amazonaws.com/role-arn: arn:aws:iam::123456789012:role/app-role
  
```

7 ROLE AND ROLEBINDING EXAMPLE (YAML)

```

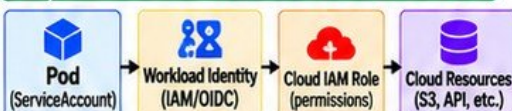
Role (namespace scoped)
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: pod-reader
  namespace: prod
rules:
- apiGroups: [""]
  resources: ["pods", "pods/log"]
  verbs: ["get", "list", "watch"]

RoleBinding
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: pod-reader-binding
  namespace: prod
subjects:
- kind: ServiceAccount
  name: app-sa
  namespace: prod
roleRef:
  kind: Role
  name: pod-reader
  apiGroup: rbac.authorization.k8s.io
  
```

8 COMMON CAUSES

Cause	Details
Missing Role/RoleBinding	ServiceAccount has no permissions
Wrong namespace	RoleBinding in different namespace
Insufficient permissions	Role does not include required verbs or resources
Using default ServiceAccount	Default SA has limited permissions
Token not mounted	automountServiceAccountToken: false
Workload identity misconfigured	Incorrect IAM/OIDC configuration
ClusterRole vs Role confusion	Using Role when ClusterRole is required
API group mismatch	Wrong API group in rules (e.g. apps vs core)
Resource name mismatch	Incorrect resource or resource name
RBAC changes not applied	Cache or propagation delay

9 WORKLOAD IDENTITY (CLOUD EXAMPLE)



Look for:

- Correct ServiceAccount annotations
- IAM/OIDC provider configured
- Correct IAM role and trust policy
- Pod is using the expected identity
- Access to cloud resources is working

10 TROUBLESHOOTING STEPS

- 1 Check ServiceAccount and token
- 2 Verify Role/RoleBinding or ClusterRoleBinding
- 3 Test authorization with kubectl auth can-i
- 4 Review application error logs
- 5 Check namespace and API group
- 6 Validate workload identity configuration
- 7 Ensure least privilege permissions
- 8 Fix missing or incorrect RBAC rules
- 9 Retest after changes

11 FIXES AND BEST PRACTICES

- ✓ Grant least privilege permissions
- ✓ Use dedicated ServiceAccounts for applications
- ✓ Avoid using the default ServiceAccount
- ✓ Use Role for namespace level access
- ✓ Use ClusterRole only when necessary
- ✓ Validate permissions before deployment
- ✓ Use workload identity for cloud access (instead of static credentials)
- ✓ Monitor and audit RBAC changes
- ✓ Regularly review and remove unused permissions



SENIOR ENGINEER TIP

Always follow the principle of least privilege. Give only the permissions your application needs, and regularly review and audit RBAC rules and workload identities.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

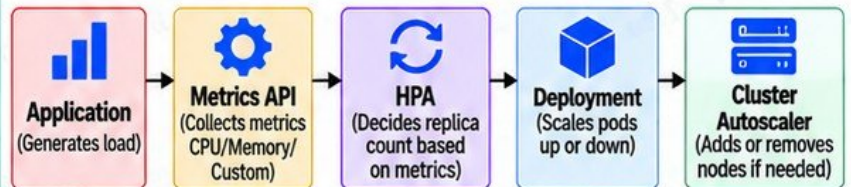
Autoscaling Failures

HPA signals, Metrics API, requests, Cluster Autoscaler, scaling lag

1 COMMON SYMPTOMS

- Pods do not scale up or down
- HPA shows no metrics or unknown values
- High CPU or memory but no scaling
- Metrics API not available
- Cluster Autoscaler not adding nodes
- Scaling is very slow or delayed
- Pods remain in Pending due to insufficient nodes
- Applications experience high latency during spikes
- HPA stuck at min or max replicas
- Unstable or frequent scaling (flapping)

2 AUTOSCALING ARCHITECTURE FLOW



If more pods are needed but there are no available nodes, Cluster Autoscaler adds new nodes. HPA scales pods based on metrics.

3 CHECK HPA STATUS

```
kubectl get hpa
kubectl describe hpa <hpa-name>
kubectl get hpa <hpa-name> -o yaml
kubectl get pods -l app=<app-name>
kubectl tpt pods -l app=<app-name>
kubectl get events --sort-by=.lastTimestamp |
grep -i hpa
```

Look for:

- Current vs desired replicas
- Target metrics and current values
- Conditions (AbleToScale, ScalingActive, ScalingLimited)
- Events and error messages
- Metrics showing unknown or missing values

4 CHECK METRICS API

```
kubectl get apiservice | grep metrics
kubectl get --raw /apis/metrics.k8s.io/v1beta1/
| jq .
kubectl top nodes
kubectl top pods
kubectl describe apiservice v1beta1.metrics.k8s.io
```

Look for:

- Metrics API is available and in a healthy state
- No errors in apiservice status
- Metrics are being returned for nodes and pods

5 CHECK CLUSTER AUTOSCALER

```
kubectl -n kube-system get pods | grep cluster-autoscaler
kubectl -n kube-system logs <cluster-autoscaler-pod>
kubectl describe node
kubectl get nodes -o wide
kubectl get events --sort-by=.lastTimestamp |
grep -i cluster-autoscaler
```

Look for:

- Cluster Autoscaler is running
- and healthy
- Nodes are being added when needed
- No cloud provider or IAM permission errors
- Scheduling failures due to insufficient nodes
- Events showing scaling decisions

6 VERIFY RESOURCE REQUESTS

```
kubectl get deployment <deploy-name> -o yaml |
grep -A 5 resources
kubectl describe pod <pod-name> | grep -A 10 "Requests:"
kubectl top pod <pod-name>
kubectl top node
```

Look for:

- CPU and memory requests are set correctly
- Requests match workload usage
- HPA uses resource metrics based on requests

7 COMMON CAUSES AND SOLUTIONS

Cause	Why it happens	How to fix
Metrics API not available	Metrics server not installed or unhealthy	Install or fix metrics server and check apiservice status
Incorrect resource requests	HPA uses requests for CPU and memory scaling	Set realistic requests and limits
HPA using wrong metric	Metric name or type is incorrect	Verify metric type and configuration
Cluster Autoscaler not scaling	IAM, cloud provider or configuration issue	Check logs, permissions and node group configuration
Insufficient cluster resources	No available nodes to schedule pods	Allow Cluster Autoscaler to add nodes or increase node group size
Scaling lag	Metrics collection and node provisioning take time	Expected delay, tune stabilization window and use predictive scaling
HPA stuck at min or max	Misconfigured min/max values	Adjust minReplicas and maxReplicas
Custom metrics not working	Prometheus Adapter or custom metrics issue	Verify custom metrics setup and API

8 EXAMPLE HPA CONFIGURATION

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: web-app-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: web-app
  minReplicas: 2
  maxReplicas: 10
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 70
```

Key points:

- HPA scales based on CPU utilization in this example
- Min 2 and max 10 replicas
- Uses resource requests to calculate utilization
- Make sure metrics are available from Metrics API

9 TROUBLESHOOTING STEPS

- 1 Check HPA status and events
- 2 Verify Metrics API is working
- 3 Confirm resource requests and actual usage
- 4 Check Cluster Autoscaler logs and node scaling
- 5 Look for scheduling failures and Pending pods
- 6 Review cloud provider limits and quotas
- 7 Validate HPA and deployment configuration
- 8 Monitor metrics, scaling behavior and timing

10 EXAMPLE HPA OUTPUT (ISSUE)

```
$ kubectl get hpa web-app-hpa
NAME              REFERENCE              TARGETS      MINPODS  MAXPODS  REPLICAS  AGE
web-app-hpa       Deployment/web-app      <unknown>/70%  2         10        2          5m
$ kubectl describe hpa web-app-hpa
Conditions:
  Type             Status  Reason
  --             --
  AbleToScale      True    ReadyForNewScale
  ScalingActive    False   FailedGetMetrics
  ScalingLimited   True    TooFewReplicas
Message: recommended size matches current size
unable to compute replica count: ;
metrics not available
the desired replica count is less
than the minimum replica count
```

11 FIXES AND BEST PRACTICES

- ✓ Ensure Metrics API is installed and healthy
- ✓ Set realistic CPU and memory requests
- ✓ Use appropriate min and max replicas
- ✓ Verify custom metrics configuration
- ✓ Ensure Cluster Autoscaler is running with correct permissions
- ✓ Monitor events and logs regularly
- ✓ Allow time for scaling (expect some lag)
- ✓ Use multiple metrics for better scaling
- ✓ Test scaling under real load

★ SENIOR ENGINEER TIP

Autoscaling relies on accurate metrics and correct resource requests. Always monitor HPA, Metrics API and Cluster Autoscaler together, and plan for scaling lag during sudden traffic spikes.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

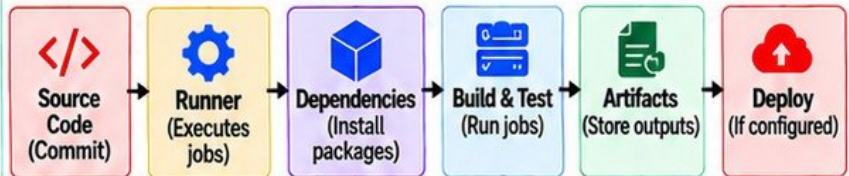
CI Pipeline Failures

Runner health, dependency failures, credentials, artifacts, reproducibility

1 COMMON SYMPTOMS

- Pipeline fails or gets stuck
- Runner is offline or not picking jobs
- Dependency or package installation fails
- Authentication or credential errors
- Build or test steps fail
- Artifacts are missing or not uploaded
- Inconsistent results between runs
- Slow pipeline or timeout
- Pipeline works on one branch but fails on another
- Environment differences cause failures

2 CI PIPELINE FLOW (WHERE FAILURES HAPPEN)



A failure can happen at any stage: runner, dependencies, credentials, build, tests, artifacts or deployment.

3 CHECK RUNNER HEALTH

```
# GitHub Actions (check runner status)
# Self-hosted runners
./config.sh --check
./svc.sh status
# GitLab Runner
gitlab-runner status
gitlab-runner verify
# List runners (GitHub)
gh run list --limit 10
```

Look for:

- Runner is online and healthy
- Runner has correct labels/tags
- Runner has enough CPU, memory and disk space
- Runner can access required networks and registries
- No frequent disconnections
- Correct runner version

4 CHECK DEPENDENCY ISSUES

```
# Example: npm
npm install
# Example: Python
pip install -r requirements.txt
# Example: Docker
docker build -t app:latest .
# Check for network/DNS issues
ping registry.npmjs.org
curl -I https://pypi.org
```

Look for:

- Package not found errors
- Version conflicts
- Network or DNS failures
- Private registry access issues
- Rate limiting
- Incompatible dependencies
- Lock file differences (e.g. package-lock.json, requirements.txt)

5 CHECK CREDENTIALS AND SECRETS

```
# Check if secrets are available
echo $AWS_ACCESS_KEY_ID
echo $DOCKER_USERNAME
# Test authentication (example)
aws sts get-caller-identity
docker login -u $DOCKER_USERNAME
# GitHub CLI (check permissions)
gh auth status
```

Look for:

- Secrets are correctly configured
- No expired credentials
- Correct permissions for the action
- Environment variables are loaded
- Access to cloud providers, container registries and internal systems
- No hardcoded secrets in code

6 CHECK BUILD, TEST AND ARTIFACTS

```
# Check build logs
docker build --progress=plain .
# Run tests locally (same as CI)
npm test
pytest
# Check artifacts (GitHub)
gh run download <run-id>
# Check artifacts size and contents
unzip artifact.zip
```

Look for:

- Build or test failures
- Missing or corrupted artifacts
- Incorrect artifact paths
- Artifacts not uploaded
- Large artifact size limits
- Test environment differences
- Build works locally but fails in CI

7 COMMON CAUSES AND FIXES

Cause	What it looks like	How to fix
Runner offline	Jobs stuck in queued	Start or fix runner, check registration
Dependency failure	Package not found	Check version, registry access, network
Credential error	401 or permission denied	Verify secrets and permissions
Build failure	Compilation or lint errors	Check logs and fix code/config
Test failure	Unit or integration tests fail	Check test logs and dependencies
Artifact upload failed	No artifacts found	Verify artifact path and permissions
Timeout	Job cancelled due to time	Increase timeout or optimize job
Inconsistent results	Works on one branch only	Check environment and dependency versions
Network/DNS issue	Cannot reach registry	Check network, proxy and DNS
Disk space issue	No space left on device	Increase disk space or clean cache

8 EXAMPLE FAILED PIPELINE LOG

```
Running with gitlab-runner 16.8.0
Fetching changes...
Installing dependencies...
npm ERR! code EAI_AGAIN
npm ERR! request to https://registry.npmjs.org/ failed
Retrying...
npm ERR! network timeout
Error: Process completed with exit code 1.
Uploading artifacts...
ERROR: No files to upload
Job failed: exit code 1
```

9 TROUBLESHOOTING STEPS

- 1 Check pipeline logs from the failing step
- 2 Verify runner health and connectivity
- 3 Confirm dependencies and versions
- 4 Validate credentials and permissions
- 5 Reproduce the issue locally
- 6 Check network, DNS and proxy settings
- 7 Verify artifact paths and storage
- 8 Compare environment variables and configs
- 9 Increase logging and enable debug mode
- 10 Fix the issue and rerun the pipeline

10 BEST PRACTICES

- ✓ Use pinned dependency versions
- ✓ Cache dependencies to speed up builds
- ✓ Use reproducible builds (lock files)
- ✓ Store and manage secrets securely
- ✓ Set reasonable timeouts
- ✓ Use clear and useful logging
- ✓ Validate artifacts after upload
- ✓ Use the same environment locally and in CI
- ✓ Monitor runner health and resource usage
- ✓ Add retries for unstable network steps

11 REPRODUCIBILITY TIPS

- ✓ Use lock files (package-lock.json, requirements.txt, go.sum, etc.)
- ✓ Specify exact version tags for base images
- ✓ Use the same OS and tool versions as CI
- ✓ Avoid latest tags in production builds
- ✓ Document environment variables required
- ✓ Use containerized builds for consistency
- ✓ Keep pipeline configuration in version control
- ✓ Test changes in a clean environment

★ SENIOR ENGINEER TIP

Make your CI pipeline deterministic and reproducible. Control versions, manage secrets securely, use caching and clear logging to quickly identify and fix failures.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

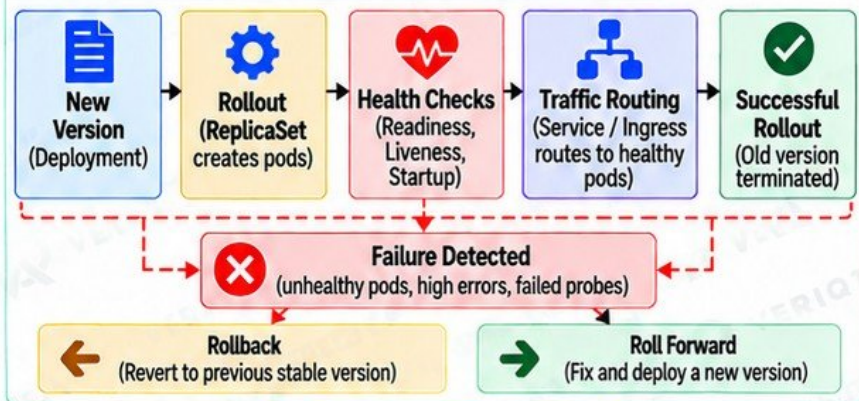
Deployment Failed in Production

Progressive delivery, health signals, rollback vs roll-forward

1 COMMON SYMPTOMS

- New version causes application errors
- Pods are not becoming Ready
- Readiness or liveness probe failures
- High error rate after deployment
- Increased latency or timeouts
- Partial or full outage
- Rollout gets stuck (ProgressDeadlineExceeded)
- New pods keep restarting (CrashLoopBackOff)
- Service still routing to unhealthy pods
- Configuration or dependency issues
- Need to rollback or roll forward

2 DEPLOYMENT FLOW (WITH HEALTH SIGNALS)



3 CHECK DEPLOYMENT STATUS

```

kubectl get deployments
kubectl describe deployment <name>
kubectl rollout status deployment <name>
kubectl get pods -l app=<name>
kubectl get events --sort-by=.lastTimestamp
kubectl logs <pod-name>
kubectl get rs -l app=<name>
  
```

Look for:

- Desired vs Available replicas
- Updated replicas count
- Events and error messages
- ProgressDeadlineExceeded
- ReplicaSet status
- Pod restart count

4 CHECK POD HEALTH

```

kubectl get pods -l app=<name>
kubectl describe pod <pod-name>
kubectl logs <pod-name> --previous
kubectl get events --field-selector  
involvedObject.name=<pod-name>
kubectl get pods -l app=<name> -o wide
  
```

Look for:

- Readiness / liveness probe failures
- CrashLoopBackOff
- Image pull errors
- Configuration errors
- Resource limits (CPU / memory)
- Application errors in logs

5 CHECK SERVICE AND TRAFFIC

```

kubectl get svc -l app=<name>
kubectl describe svc <service-name>
kubectl get endpoints <service-name>
kubectl get ingress
kubectl describe ingress <ingress-name>
  
```

Look for:

- Service has healthy endpoints
- Traffic is routing to correct pods
- Ingress rules and annotations
- No 5xx errors at the load balancer
- Session affinity or routing issues

6 CHECK HEALTH SIGNALS (OBSERVABILITY)

```

kubectl top pods -l app=<name>
kubectl top nodes
kubectl get hpa
# Check logs and metrics (Prometheus/Grafana)
# Check APM or application monitoring
# Check error rates, latency, and traffic
  
```

Look for:

- High error rate (5xx)
- Increased latency
- High CPU or memory usage
- HPA scaling events
- Dependency failures (database, external APIs)
- Anomalies in metrics or logs

7 PROGRESSIVE DELIVERY OPTIONS

1. Rolling Update (default)	2. Canary Deployment	3. Blue-Green Deployment	4. Feature Flags
• Gradually replace pods with new version • Minimize downtime • Monitor health signals during rollout	• Send small % of traffic to new version • Monitor metrics and errors • Increase traffic gradually if healthy	• Deploy new version in parallel environment • Switch traffic when healthy • Quick rollback if needed	• Deploy code with feature disabled • Enable gradually • Reduce risk with controlled release

8 ROLLBACK VS ROLL-FORWARD

Rollback	Roll Forward
• Use when new version is unstable • Quickly revert to previous version • Works if previous version is still working • Minimizes downtime • Use: <code>kubectl rollout undo deployment <name></code>	• Use when issue is fixable quickly • Deploy a new version with fix • Better when database changes are involved • Avoids long rollback cycles • Use: fix the issue, build and deploy a new version

9 COMMON CAUSES

Cause	Details
Application error	Bug in new code, misconfiguration
Failed health checks	Readiness or liveness probe failing
Image or dependency issue	Missing packages, incompatible version
Resource limits	Insufficient CPU or memory
Database migration issue	Breaking schema changes
External dependency failure	Third party API or service unavailable
Configuration error	Wrong environment variables or secrets
Traffic spike	New version cannot handle the load
Progress deadline exceeded	Rollout did not complete in time

10 EXAMPLE ROLLBACK

```

# Check rollout history
kubectl rollout history deployment <name>

# Rollback to previous version
kubectl rollout undo deployment <name>

# Rollback to a specific revision
kubectl rollout undo deployment <name> --to-revision=2

# Verify rollout
kubectl rollout status deployment <name>
  
```

11 BEST PRACTICES

- ✓ Use readiness and liveness probes
- ✓ Monitor error rates, latency and key metrics
- ✓ Use progressive delivery (canary or blue-green)
- ✓ Automate health checks and rollback
- ✓ Set resource requests and limits
- ✓ Test in staging environment
- ✓ Use feature flags for risky changes
- ✓ Keep previous version available for quick rollback
- ✓ Monitor dependencies and external services
- ✓ Have a clear rollback and roll-forward plan

★ SENIOR ENGINEER TIP

Always monitor health signals during and after deployment. Use progressive delivery to reduce risk, and be ready to rollback or roll forward based on real data, not assumptions.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

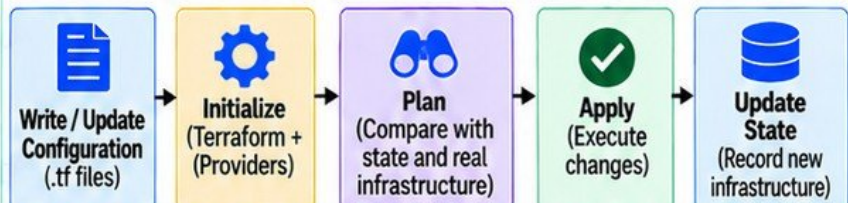
Terraform Failure & State Problems

Drift, locks, replacement, state integrity, provider/API failures

1 COMMON SYMPTOMS

- Plan shows unexpected changes (drift)
- Resources want to be replaced
- State file is locked
- Apply fails with provider or API errors
- State becomes inconsistent or corrupted
- Resources are missing from state
- Partial apply (some resources created)
- Permission or authentication failures
- Rate limiting or quota errors
- Timeouts or network issues
- Configuration changes cause mass replacement
- Workspaces or backend misconfiguration

2 TERRAFORM EXECUTION FLOW



Failures can happen at any stage:

- Provider/API errors
- State lock issues
- Configuration drift
- Authentication/permissions
- Network/timeouts
- Resource replacement
- Partial apply
- State corruption
- Rate limits/quotas

3 CHECK CURRENT STATE

```

terraform version
terraform init
terraform workspace list
terraform workspace show
terraform state list
terraform state show <resource>
terraform plan
terraform refresh
  
```

Look for:

- Correct Terraform version
- Correct backend and workspace
- Resources in state match real infra
- Unexpected changes in plan
- Missing resources in state
- Provider or backend errors
- Initialization issues

4 CHECK STATE AND LOCKS

```

terraform state list
terraform state show <resource>
terraform state pull >
state-backup.tfstate
terraform force-unlock <lock-id>
terraform state rm <resource>
terraform state mv <old> <new>
  
```

Look for:

- State file is locked
- State lock (no active run)
- Corrupted or incomplete state
- Resources present in real infra but missing from state
- Incorrect resource addressing
- Safe state recovery options

5 CHECK DRIFT AND REPLACEMENT

```

terraform plan
terraform plan -refresh-only
terraform apply -refresh-only
terraform show
terraform state show <resource>
  
```

Look for:

- Unexpected changes (drift)
- Resources marked for replacement
- Changes outside of Terraform
- Configuration changes causing recreation
- Force new value (immutable fields)
- Default values or provider behavior changes

6 CHECK PROVIDER AND API ISSUES

```

terraform init -upgrade
terraform providers
terraform plan
TF_LOG=DEBUG terraform apply
terraform validate
  
```

Look for:

- Provider version compatibility
- Authentication and credentials
- API errors (rate limits, quotas)
- Network connectivity issues
- Timeouts
- Deprecated resources or arguments
- Cloud service outages

7 COMMON ERRORS AND CAUSES

Error Message	Cause	How to Fix
Error acquiring the state lock	State is locked by another run or stale lock	Check active runs and force-unlock if safe
Provider API error	Authentication, permissions, rate limits, or service outage	Verify credentials, permissions, quotas and service status
Plan wants to replace resource	Immutable field changed or configuration drift	Review changes, confirm if replacement is expected, or adjust configuration
Resource already exists	Resource exists outside of Terraform or in another state	Import the existing resource terraform import <address> <id>
State is inconsistent	Manual changes, partial apply, or state corruption	Refresh, fix state, or restore from backup
Timeout or network error	Network, firewall or provider issue	Check connectivity, retries and provider status
Unsupported argument	Provider version mismatch	Upgrade provider or fix configuration

8 EXAMPLE PLAN SHOWING REPLACEMENT

```

# terraform plan
-/+ resource "aws_instance" "web" {
  ~ ami           = "ami-12345678" -> "ami-abcdef12" # forces replacement
  ~ instance_type = "t3.micro" -> "t3.small"
  ~ tags = {
    ~ "Name" = "web-v1" -> "web-v2"
  }
}
Plan: 1 to add, 0 to change, 1 to destroy.
# Resource will be replaced due to immutable field change.
  
```

9 TROUBLESHOOTING STEPS

- Check error message and stage (init, plan, apply)
- Verify state lock and remove if stale
- Validate credentials and permissions
- Check provider version and upgrade if needed
- Run plan with refresh-only to check drift
- Review changes for unexpected replacements
- Check network connectivity and API status
- Verify workspace, backend and state configuration
- Import or fix resources if state is inconsistent
- Re-run plan and apply after fixing the issue

10 BEST PRACTICES

- Use remote state with locking
- Enable state backups
- Use version control for .tf files
- Use workspaces for environments
- Review plan output carefully
- Use least privilege access
- Monitor provider and cloud service status
- Set timeouts and retries
- Avoid manual changes in the console
- Keep providers up to date

★ SENIOR ENGINEER TIP

Always review plan output, understand why resources will be replaced, and verify state integrity before applying changes to production.

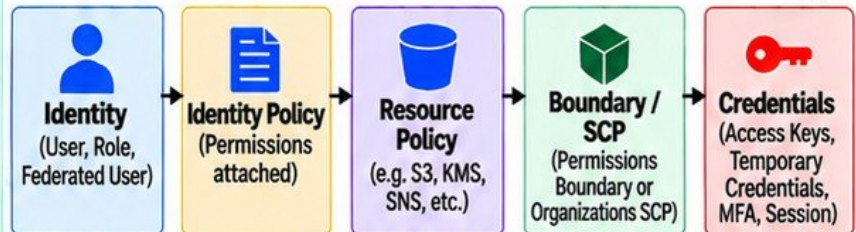
Cloud IAM Access Failures

Identity → policy → resource policy → boundary/SCP → credentials

1 COMMON SYMPTOMS

- Access denied errors (403)
- User or role cannot perform an action
- Works for some resources but not others
- Works in one account but fails in another
- AssumeRole or federation issues
- Temporary credentials expired
- MFA or session policy restrictions
- Explicit deny in policy, boundary or SCP
- Resource policy blocking access
- Permissions work in console but not in CLI/SDK
- Unexpected cross-account access failure

2 IAM ACCESS EVALUATION FLOW



All policies are evaluated together. Explicit Deny always wins.

3 CHECK IDENTITY AND CREDENTIALS

```
aws sts get-caller-identity
aws sts get-session-token
aws sts assume-role --role-arn <role-arn>
--role-session-name test
aws configure list
aws iam get-user
aws iam get-role --role-name <role-name>
```

Look for:

- Correct account and identity
- Correct role assumed
- Valid and not expired credentials
- MFA required
- Session policy restrictions
- Correct region and profile

4 CHECK IDENTITY POLICIES

```
aws iam list-attached-user-policies
--user-name <user>
aws iam list-attached-role-policies
--role-name <role>
aws iam list-user-policies --user-name <user>
aws iam get-policy-version --policy-arn <arn>
--version-id <id>
```

Look for:

- Required permissions are allowed
- No explicit deny
- Correct action, resource and condition
- Policy attached to the correct user or role
- Missing permissions

5 CHECK RESOURCE POLICIES

```
aws s3api get-bucket-policy --bucket <bucket>
aws kms get-key-policy --key-id <key-id>
aws sns get-topic-attributes --topic-arn <arn>
aws lambda get-policy --function-name <name>
aws iam get-role --role-name <role>
| grep AssumeRolePolicyDocument -A 10
```

Look for:

- Resource policy allows your identity or role
- Correct principal (account, role, user)
- Conditions (IP, VPC, source account, etc.)
- Cross-account permissions
- Missing or restrictive conditions

6 CHECK BOUNDARIES AND SCPs

```
aws iam get-role --role-name <role>
| grep PermissionsBoundary -A 5
aws organizations describe-effective-policy
--policy-type SERVICE_CONTROL_POLICY
--policy-id <id>
aws organizations list-policies
--filter SERVICE_CONTROL_POLICY
```

Look for:

- Permissions boundary restricting permissions
- SCPs blocking actions
- Explicit deny in SCP
- Account level restrictions
- Service control policies from parent OU

7 COMMON ACCESS DENIED ERRORS

Error Message	Meaning	Common Cause	How to Fix
AccessDenied	Not authorized	Missing permissions	Add required permissions
User is not authorized to perform: s3:ListBucket	Action not allowed	Identity policy or resource policy	Update policy or resource policy
Explicit deny	Explicitly blocked	SCP or policy deny	Remove or adjust explicit deny
Not authorized to assume role	Cannot assume role	Trust policy issue	Update role trust policy
ExpiredToken	Credentials expired	Temporary credentials expired	Refresh credentials
InvalidClientTokenId	Invalid credentials	Wrong or deleted keys	Update AWS credentials
AccessDeniedException (KMS)	KMS access denied	Missing kms:Decrypt or key policy	Update key policy and IAM policy

8 EXAMPLE POLICY (LEAST PRIVILEGE)

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "s3:GetObject",
        "s3:ListBucket"
      ],
      "Resource": [
        "arn:aws:s3::my-bucket",
        "arn:aws:s3::my-bucket/*"
      ]
    }
  ]
}
```

Key points:

- Allow only required actions
- Use specific resources
- Use conditions where possible (e.g. IP, VPC, MFA)
- Avoid wildcard *
- Review and remove unused permissions
- Follow least privilege principle

9 TROUBLESHOOTING STEPS

- Confirm who you are (get-caller-identity)
- Verify credentials are valid and not expired
- Check the exact error message
- Check identity policies for required permissions
- Check resource policy (e.g. S3, KMS, SNS, etc.)

- Check permissions boundary if attached
- Check organization SCPs
- Review conditions (region, VPC, IP, etc.)
- Test with minimal permissions
- Fix and retest the access

10 BEST PRACTICES

- Use least privilege permissions
- Use roles instead of long-lived access keys
- Enable MFA for users
- Use conditions (IP, VPC, source account, etc.)
- Regularly review and remove unused permissions
- Monitor access with CloudTrail
- Use AWS Organizations SCPs for guardrails
- Use temporary credentials and rotate keys

★ SENIOR ENGINEER TIP

When troubleshooting access issues, always check for explicit deny, verify the full policy chain (identity, resource, boundary/SCP, and conditions), and use least privilege permissions.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

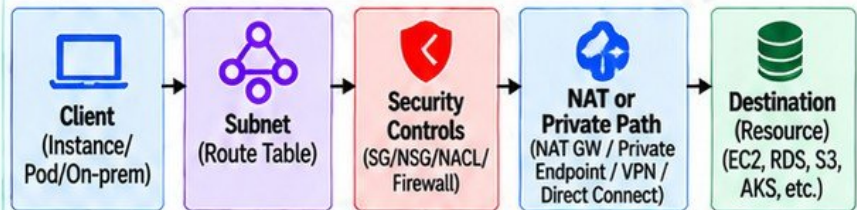
Cloud Network Connectivity Failures

VPC/VNet path, routes, security controls, NAT, private endpoints

1 COMMON SYMPTOMS

- Cannot connect to a resource (timeout)
- Connection refused
- DNS resolution fails
- Works in one subnet but not another
- Cannot reach internet (no NAT)
- Cannot reach private endpoint
- Traffic blocked by security group/NSG
- Intermittent connectivity
- Application cannot reach database or API
- Cross-account or cross-VPC/VNet access fails

2 CLOUD NETWORK PATH (END TO END)



Traffic must have a valid path, allowed by all security controls, and a working egress/ingress route to reach the destination.

3 CHECK BASIC CONNECTIVITY

```

ping <destination-ip>
curl -v <url>
nc -vz <host> <port>
tracert <destination-ip>
nslookup <domain>
dig <domain>
telnet <host> <port>
  
```

Look for:

- Can you resolve the name?
- Is the port open?
- Where does the traffic stop?
- Is it a DNS, routing or security issue?
- Does it work from another subnet or instance?

4 CHECK ROUTES (VPC/VNET PATH)

```

aws ec2 describe-route-tables
--filters "Name=vpc-id,Values=<vpc-id>"
az network route-table show
--name <route-table> --resource-group <rg>
ip route # Linux
route print # Windows
  
```

Look for:

- Correct route to destination (IGW, NAT, TGW, VPN, etc.)
- Blackhole or missing route
- Overlapping routes
- Incorrect next hop
- Subnets associated with the right route table

5 CHECK SECURITY CONTROLS

```

# AWS Security Groups
aws ec2 describe-security-groups
--group-ids <sg-id>
# Azure NSG
az network nsg show --name <nsg>
# Network ACLs
aws ec2 describe-network-acls
--filters "Name=vpc-id,Values=<vpc-id>"
  
```

Look for:

- Inbound and outbound rules
- Correct port and protocol
- Source and destination CIDR
- NACL allow/deny rules
- Firewall or cloud security appliance rules
- Rules in both directions (client → server, server → client)

6 CHECK NAT, INTERNET AND PRIVATE ENDPOINTS

```

# Check NAT Gateway (AWS)
aws ec2 describe-nat-gateways
# Check NAT Gateway (Azure)
az network nat gateway show --name <name>
# Check Private Endpoint (Azure)
az network private-endpoint show --name <name>
# Test internet connectivity
curl -I https://www.google.com
  
```

Look for:

- NAT gateway is available and has bandwidth
- Subnet uses the correct route to NAT
- Private endpoint is approved and connected
- DNS resolves to private IP
- No firewall blocking egress
- Service endpoint or private link configured

7 COMMON CAUSES AND FIXES

Cause	What it looks like	How to fix
DNS issue	Cannot resolve name	Check DNS settings, use correct DNS server or private DNS
Route missing	Traffic goes to wrong place or blackhole	Update route table with correct next hop
Security group/NSG block	Connection timeout or refused	Allow correct port, protocol and CIDR
NACL block	Traffic blocked at subnet level	Update NACL rules
No NAT (private subnet)	Cannot reach internet	Configure NAT gateway or egress solution
Private endpoint issue	Cannot reach service	Check private endpoint status and DNS resolution
Firewall or proxy block	Intermittent or consistent failure	Update firewall/proxy rules
Cross-VPC/VNet issue	Cannot reach resources in another VPC	Configure peering, TGW, VPN or Private Link
Service not listening	Timeout or connection refused	Verify service is running and listening on correct port

8 EXAMPLE TROUBLESHOOTING FLOW

- 1 → Test from source
ping, curl, traceroute, nslookup
- 2 → Check DNS
Resolve name and confirm IP
- 3 → Check route table
Ensure correct next hop
- 4 → Check security controls
SG/NSG/NACL/Firewall rules
- 5 → Verify NAT or private endpoint
Check egress/ingress configuration
- 6 → Test again
Confirm connectivity from source to destination

9 REAL WORLD EXAMPLE

Scenario: An application in a private subnet cannot access an external API.

Troubleshooting:

- DNS resolves correctly
- Route table has no route to internet
- No NAT gateway configured
- Traffic cannot leave the VPC
- Fix: Add NAT gateway and update route table
- Result: Application can now access the API

10 PRODUCTION TIPS

- ✓ Use clear and consistent CIDR planning
- ✓ Document all routes and security rules
- ✓ Use private endpoints for cloud services
- ✓ Monitor VPC flow logs / NSG flow logs
- ✓ Use DNS and route visualization tools
- ✓ Test connectivity during deployments
- ✓ Use infrastructure as code for networking
- ✓ Apply least privilege access
- ✓ Implement centralized network monitoring
- ✓ Regularly review and clean up unused rules

11 USEFUL TOOLS AND LOGS

- ✓ VPC Flow Logs / NSG Flow Logs
- ✓ CloudTrail / Activity Logs
- ✓ Network Watcher (Azure)
- ✓ VPC Reachability Analyzer (AWS)
- ✓ Security Group/NACL/Firewall Logs
- ✓ Packet capture (tcpdump)
- ✓ Cloud monitoring and alerting
- ✓ Service logs (e.g. RDS, AKS, etc.)
- ✓ DNS query logs
- ✓ Traceroute and mtr for path analysis

★ SENIOR ENGINEER TIP

When troubleshooting cloud network connectivity, always verify DNS first, then routes, then security controls, then NAT or private endpoints. Test from the source and follow the traffic path.

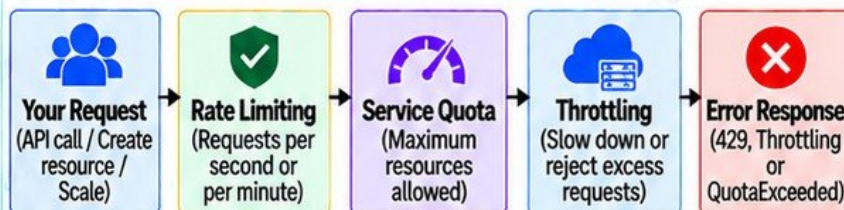
Cloud Quotas & API Throttling

Rate limits, quota exhaustion, retry behavior, scaling constraints

1 COMMON SYMPTOMS

- API rate limit exceeded errors (429, Throttling)
- Quota exceeded errors
- Resource creation fails
- Scaling or auto scaling stops working
- Intermittent failures during high load
- Some regions work but others fail
- Operations are delayed or slow
- CI/CD or automation jobs fail
- Third party APIs return rate limit errors
- Services become unstable during spikes

2 HOW QUOTAS AND THROTTLING WORK



Each cloud provider enforces rate limits and service quotas per account, per region and sometimes per resource type.

3 CHECK ERROR MESSAGES

Common AWS errors

An error occurred (Throttling) when calling the DescribeInstances operation: Rate exceeded

An error occurred (ServiceQuotaExceeded) when calling the RunInstances operation

Look for:

- Throttling or rate limit errors
- Quota exceeded errors
- Which API and resource type
- Request ID for troubleshooting
- Region and account limits
- Whether the issue is temporary or a hard quota limit

4 CHECK SERVICE QUOTAS

AWS Service Quotas

```
aws service-quotas list-service-quotas --service-code ec2
aws service-quotas get-service-quota --service-code ec2 \
--quota-code L-1216C47A # example
aws ec2 describe-account-attributes
```

Look for:

- Current quota values
- Usage vs limit
- Regional differences
- Adjustable vs non-adjustable quotas
- Pending quota increase requests
- Which accounts and regions are affected

5 CHECK API RATE LIMITS

Example: Check API rate limits
(varies by provider and service)

AWS (general guidance)
aws sts get-caller-identity

Check CloudWatch for throttling
aws cloudwatch get-metric-statistics --namespace AWS/Usage \ --metric-name CallCount

Look for:

- API call rate limits per service
- Throttling metrics in monitoring
- Peak request rates during failures
- Specific APIs being throttled
- Client side retry behavior
- Whether multiple services are affected

6 CHECK SCALING CONSTRAINTS

Check instance type limits (AWS)
aws ec2 describe-instance-type-offerings --location-type region \ --region us-east-1 \ --filters Name=instance-type, \ Values=t3.large

Check auto scaling limits
aws autoscaling describe-account-limits

Look for:

- Instance type availability and limits
- Auto Scaling group limits
- vCPU and instance quotas
- Regional capacity constraints
- Limits for load balancers, EIPs, subnets, etc.
- Whether the desired scale exceeds quota or capacity

7 COMMON CAUSES AND FIXES

Cause	What it looks like	How to fix
API rate limiting	429 or Throttling errors	Implement retry with exponential backoff and jitter
Service quota exceeded	QuotaExceeded error	Request a quota increase
Scaling too fast	Auto Scaling fails or is slow	Use gradual scaling and warm up
Too many parallel requests	Intermittent API failures	Reduce concurrency and use queueing
Regional capacity limits	Cannot launch resources in a region	Use another region or different instance type
Automation loops	Repeated failed requests	Add retry logic and circuit breakers
Third party API limits	Rate limit exceeded	Use caching, request batching or higher plan limits
Misconfigured quota	Lower than expected limit	Check and update service quotas
Burst traffic	Works normally but fails during spikes	Use request throttling and load smoothing

8 EXAMPLE: RETRY WITH EXPONENTIAL BACKOFF

```
import time
import random

max_retries = 5
base_delay = 1

for attempt in range(max_retries):
    try:
        # API call (example)
        response = call_cloud_api()
        break
    except ThrottlingError:
        delay = base_delay * (2 ** attempt)
        delay = delay + random.uniform(0, 1)
        print(f"Throttled. Retrying in {delay:.2f}s...")
        time.sleep(delay)
```

9 TROUBLESHOOTING STEPS

- 1 Identify the exact error and API
- 2 Check current quota and usage
- 3 Review recent changes and traffic patterns
- 4 Verify region and account limits
- 5 Check monitoring and throttling metrics
- 6 Reduce request rate or concurrency
- 7 Implement retry with exponential backoff
- 8 Request quota increase if needed
- 9 Try a different region or instance type
- 10 Test again and monitor for stability

10 BEST PRACTICES

- ✓ Implement retry with exponential backoff
- ✓ Use request throttling and queueing
- ✓ Monitor API error rates and quotas
- ✓ Request quota increases proactively
- ✓ Design for graceful degradation
- ✓ Use multi region and fallback options

11 USEFUL TOOLS AND LOGS

- ✓ CloudWatch (AWS)
- ✓ Azure Monitor
- ✓ GCP Cloud Monitoring
- ✓ CloudTrail / Activity Logs
- ✓ Service Quotas Dashboard
- ✓ SDK retry configuration
- ✓ Third party API dashboards

★ SENIOR ENGINEER TIP

Watch for quota limits and throttling before scaling. Implement retries, monitor usage, and request quota increases early to avoid production impact.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

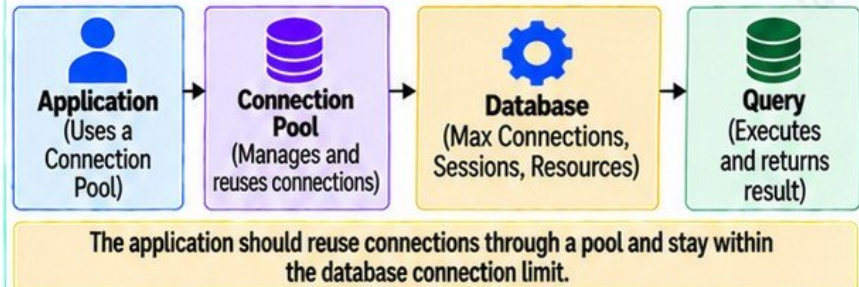
Database Connection Exhaustion

Pools, connection limits, slow queries, leaks, timeout propagation

1 COMMON SYMPTOMS

- Too many connections errors
- Cannot acquire connection from pool
- Application response time increases
- Database becomes slow or unresponsive
- New connections are rejected
- Intermittent timeouts
- Connection pool exhausted
- High number of idle or sleeping connections
- Long running or blocked queries
- Memory and CPU increase on database server
- Cascading failures in dependent services

2 HOW DATABASE CONNECTIONS WORK



3 CHECK CURRENT CONNECTIONS

```
# PostgreSQL
SELECT count(*) FROM pg_stat_activity;
SELECT * FROM pg_stat_activity
ORDER BY state, query_start;
SELECT * FROM pg_stat_activity
WHERE state = 'active';
```

Look for:

- Total active connections
- Idle connections
- Long running queries
- Blocked or waiting queries
- Connections from each service
- Unusual or sudden increase
- Connections in idle transaction

4 CHECK CONNECTION LIMITS

```
# PostgreSQL
SHOW max_connections;
# MySQL
SHOW VARIABLES LIKE 'max_connections';
# Check current usage (PostgreSQL)
SELECT count(*) AS current_connections
FROM pg_stat_activity;
```

Look for:

- Maximum allowed connections
- Current connection usage
- How close you are to the limit
- Reserved connections for admin
- Connection limit per user or service

5 CHECK CONNECTION POOL SETTINGS

```
# Example (application configuration)
maxPoolSize = 20
minIdle = 5
idleTimeout = 30000 # 30 seconds
connectionTimeout = 10000 # 10 seconds
leakDetectionThreshold = 60000
```

Look for:

- Pool size too large or too small
- Idle timeout settings
- Connection timeout settings
- Leak detection enabled
- Pool not releasing connections
- Separate pools for different services
- Whether multiple instances are creating too many connections

6 CHECK SLOW QUERIES AND BLOCKING

```
# PostgreSQL - slow queries
SELECT pid, now() - query_start AS duration,
query FROM pg_stat_activity
WHERE state = 'active'
ORDER BY duration DESC;
# PostgreSQL - blocking queries
SELECT * FROM pg_locks;
```

Look for:

- Long running queries
- Queries waiting for locks
- High I/O or CPU queries
- Missing indexes
- Table scans
- Blocking and deadlocks
- Queries holding connections for too long

7 COMMON CAUSES AND FIXES

Cause	What it looks like	How to fix
Connection leak	Connections not closed after use	Ensure connections are closed (use try finally or connection pooling)
Pool size too large	Too many connections from the application	Use appropriate pool size based on DB limits and instance count
Slow queries	Long running queries hold connections	Optimize queries, add indexes, use query timeouts
Idle connections	Many idle connections consuming resources	Set idle timeout and connection lifetime
High traffic or spikes	Sudden increase in connections	Use autoscaling, queue requests, rate limit
Database max limit reached	Too many connections error	Increase DB limit (if possible) and reduce application connections
Multiple instances	Each instance creates its own pool	Calculate total connections across all instances
Network or timeout issues	Connections stay open due to timeouts	Set proper connection and read/write timeouts

8 EXAMPLE: HANDLE TIMEOUTS PROPERLY

```
import psycopg2
from psycopg2 import pool

db_pool = psycopg2.pool.SimpleConnectionPool(
    5, 20,
    host='db-host',
    user='user',
    password='password',
    database='app',
    connect_timeout=10 # connection timeout
)

try:
    conn = db_pool.getconn()
    # Execute query with timeout
    with conn.cursor() as cur:
        cur.execute("SET statement_timeout = 5000;")
        cur.execute("SELECT * FROM users;")
finally:
    db_pool.putconn(conn) # always return to pool
```

9 TROUBLESHOOTING STEPS

- 1 Check current connection count and limits
- 2 Identify long running and blocked queries
- 3 Review application connection pool settings
- 4 Check for connection leaks in the code
- 5 Analyze traffic patterns and recent changes
- 6 Verify database logs for connection errors
- 7 Check network timeouts and failures
- 8 Optimize slow queries and add indexes
- 9 Adjust pool size and timeouts
- 10 Test after applying changes and monitor

10 BEST PRACTICES

- Use connection pooling
- Set reasonable pool size
- Always close connections
- Set connection and query timeouts
- Monitor connection usage
- Optimize queries and indexes
- Use autoscaling for high traffic

11 USEFUL TOOLS AND LOGS

- Database logs (PostgreSQL, MySQL)
- pg_stat_activity (PostgreSQL)
- Performance Insights (AWS RDS)
- Cloud Monitoring (GCP Cloud SQL)
- Azure Monitor (Azure Database)
- Application APM (Datadog, New Relic)
- Connection pool metrics
- Slow query logs and explain plans

★ SENIOR ENGINEER TIP

Use connection pools, set proper timeouts, monitor connection usage and slow queries, and design for graceful handling of spikes to prevent connection exhaustion.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

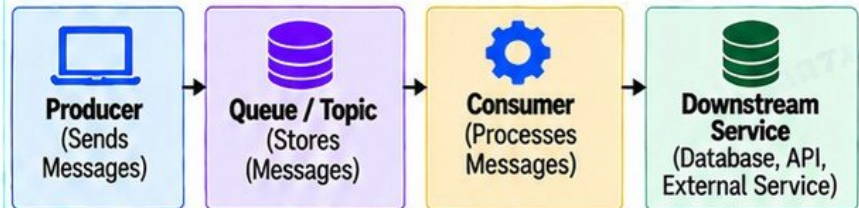
Queue Backlog & Dependency Saturation

Consumer lag, throughput mismatch, backpressure, poison messages

1 COMMON SYMPTOMS

- Growing queue backlog
- Consumer lag increases
- Messages take too long to process
- Throughput lower than expected
- High retry counts or redeliveries
- Poison messages stuck in queue
- Downstream service saturation
- Increased latency across services
- Consumers crash or restart frequently
- Memory or CPU spikes on consumers
- Messages expire or get discarded
- User-facing delays and timeouts

2 HOW MESSAGE QUEUES WORK



If consumers are slower than producers or downstream services are saturated, messages accumulate in the queue and create a backlog.

3 CHECK QUEUE BACKLOG

```

# AWS SQS
aws sqs get-queue-attributes
--queue-url <url>
--attribute-names
ApproximateNumberOfMessages
ApproximateAgeOfOldestMessage

# Azure Service Bus
az servicebus queue show
--namespace-name <ns>
--name <queue-name>
--query "messageCount,
activeMessageCount"
  
```

Look for:

- Number of messages in queue
- Age of oldest message
- Messages increasing over time
- Sudden spikes in backlog
- Failed or dead-lettered messages
- Message size and retention time
- Regional differences (in distributed systems)

4 CHECK CONSUMER LAG AND THROUGHPUT

```

# Kafka
kafka-consumer-groups.sh
--bootstrap-server <broker>
--group <group-id> --describe

# RabbitMQ
rabbitmqctl list_queues
name messages_ready
messages_unacknowledged

# Monitor consumer metrics
# (Prometheus / CloudWatch / Azure Monitor)
consumer_lag
consume_rate
process_rate
  
```

Look for:

- Consumer lag per group/partition
- Messages unacknowledged
- Consumer throughput (messages per second)
- Slow or stuck consumers
- Frequent rebalances (Kafka)
- High retry or failure rate
- Uneven partition consumption
- Consumers scaling properly

5 CHECK DOWNSTREAM DEPENDENCIES

```

# Check database performance
SELECT * FROM pg_stat_activity
WHERE state = 'active';

# Check API latency
curl -w "time:%(time_total)\n"
-o /dev/null -s <api-endpoint>

# Check service health
kubectl get pods
kubectl top pods
  
```

Look for:

- Database connection limits
- Slow queries
- High CPU or memory usage
- Increased response time
- Rate limit or throttling errors
- Service timeouts
- Downstream service saturation
- Network latency or errors
- Cascading failures

6 CHECK POISON MESSAGES

```

# AWS SQS dead-letter queue
aws sqs receive-message
--queue-url <dlq-url>
--max-number-of-messages 10

# Kafka (check failed messages)
kafka-console-consumer.sh
--bootstrap-server <broker>
--topic <dlq-topic>

# RabbitMQ (dead letter exchange)
rabbitmqctl list_queues
name messages
  
```

Look for:

- Messages that always fail
- Malformed or invalid data
- Processing errors
- Schema or format issues
- Messages exceeding size limits
- Business logic failures
- Repeated retries
- Dead-letter queue (DLQ) growth
- Fix and reprocess or discard invalid messages

7 COMMON CAUSES AND FIXES

Cause	What it looks like	How to fix
Consumer slower than producer	Backlog grows over time	Scale consumers, optimize processing, increase throughput
Downstream service slow	Consumers stuck or timing out	Fix performance issues, add caching, increase resources
Poison messages	Repeated failures and retries	Move to DLQ, fix data, reprocess valid messages
Insufficient consumer scaling	High lag, low throughput	Enable auto scaling, increase concurrency
Rate limiting or throttling	Errors from downstream APIs	Implement retries with backoff, use rate limits appropriately
Large message size	Slow processing, higher costs	Reduce message size or use object storage for large payloads
Network or connectivity issues	Intermittent failures	Check network, retries, multi-AZ-region
Malfunctioning consumer code	Crashes or high error rate	Fix bugs, improve error handling and retries

8 EXAMPLE: HANDLE BACKPRESSURE

```

import time
from queue_client import MessageQueue

while True:
    try:
        msg = queue.receive_message()
        process_message(msg)
        queue.ack(msg)
    except TransientError:
        # retry with exponential backoff
        time.sleep(2 ** retry_count)
    except Exception as e:
        # send to DLQ for poison messages
        queue.send_to_dlq(msg)
        log.error(f"Failed message: {e}")
  
```

9 TROUBLESHOOTING STEPS

- 1 Check queue depth and message age
- 2 Verify consumer lag and throughput
- 3 Check downstream service health
- 4 Identify poison messages and DLQ
- 5 Review logs and error messages
- 6 Check consumer scaling and configuration
- 7 Verify rate limits and throttling
- 8 Analyze message processing time
- 9 Test with smaller load or replay messages
- 10 Implement monitoring and alerts

10 BEST PRACTICES

- Right size consumers and enable auto scaling
- Implement backpressure and retries
- Use dead-letter queues for failed messages
- Monitor queue depth, lag and age
- Set appropriate message retention
- Optimize message size and processing
- Use idempotent consumer logic

11 USEFUL TOOLS AND LOGS

- CloudWatch (AWS SQS)
- Azure Monitor (Service Bus)
- Prometheus + Grafana
- Kafka Consumer Group Tools
- RabbitMQ Management UI
- Application logs (consumer)
- Distributed tracing (Jaeger)
- Alerting (PagerDuty, Alertmanager)

★ SENIOR ENGINEER TIP

Monitor queue depth, consumer lag and error rate. Implement backpressure, retries with exponential backoff, and dead-letter queues to handle poison messages and prevent system-wide impact.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

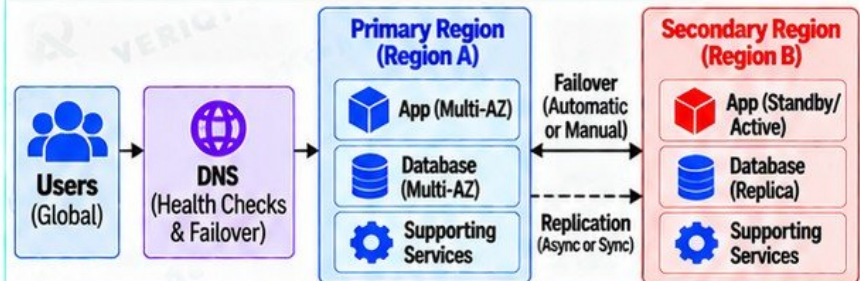
Multi-AZ / Regional Failure

Failure domains, failover, dependency mapping, RTO/RPO, degraded operation

1 COMMON SYMPTOMS

- Service unavailable in one or more AZ/region
- Increased latency or timeouts
- Partial outage (some services working)
- Traffic not failing over automatically
- Database or state store unavailable
- Cross-AZ or cross-region replication lag
- Dependency not available in secondary region
- Health checks failing in one AZ/region
- DNS still sending traffic to unhealthy region
- Service is running in degraded mode
- Increased error rates (5xx, connection errors)

2 MULTI-AZ / REGIONAL ARCHITECTURE (HIGH LEVEL)



Design for failure. Keep services in multiple AZs and have a tested regional failover plan with clear RTO, RPO and degraded operation capabilities.

3 CHECK REGION AND AZ HEALTH

```
# AWS - check availability zones
aws ec2 describe-availability-zones
--region us-east-1

# AWS - check service health
aws health describe-events
--region us-east-1

# Azure - check region status
az account list-locations
az monitor service-health list

# GCP - check zone and region status
gcloud compute zones list
--filter=region:us-central1
```

Look for:

- AZ or region marked as impaired
- Service health events
- Increased error rates in specific AZ/region
- Network or control plane issues
- Impact on dependent services
- Cross-region replication status
- DNS/traffic still routed to unhealthy region

4 CHECK FAILOVER AND DNS

```
# Check DNS records (Route 53 example)
aws route53 list-resource-record-sets \
--hosted-zone-id Z123456789

# Test which region you are hitting
curl -I https://your-domain.com

# Check health checks
aws route53 list-health-checks

# Azure Traffic Manager
az network traffic-manager profile show \
--name my-profile

# GCP Cloud DNS
gcloud dns record-sets list \
--zone=your-zone
```

Look for:

- Health check failures
- DNS still pointing to unhealthy region
- Traffic not failing over
- Incorrect routing policies
- TTL too high
- DNS propagation delay
- Users still reaching affected region
- Geographic routing misconfiguration

5 CHECK DATA REPLICATION AND RTO/RPO

```
# AWS RDS replication status
aws rds describe-db-instances \
--query 'DBInstances[*].[DBInstanceIdentifier, DBInstanceStatus, AvailabilityZone]'

# AWS S3 cross-region replication
aws s3api get-bucket-replication --bucket my-bucket

# Azure SQL geo-replication
az sql db show --name mydb --resource-group rg

# GCP Cloud SQL failover replica
gcloud sql instances describe mydb
```

Look for:

- Replication lag
- Replica in correct region
- RPO (data loss tolerance)
- RTO (time to recover)
- Replication errors
- Write/read failover readiness
- Data consistency after failover
- Backup availability in secondary region

6 CHECK DEPENDENCIES

```
# Check dependent services
# (example: check external API)
curl -I https://api.service.com

# Check VPC peering / transit
aws ec2 describe-vpc-peering-connections

# Check service endpoints (Azure)
az network private-endpoint list

# Check downstream service health
kubectl get endpoints
kubectl get pods -A
```

Look for:

- Dependencies not available in secondary region
- VPC peering / transit issues
- Private endpoints failing
- Third party API outages
- Service endpoint misconfiguration
- Cascading failures across services
- Hidden single-region dependencies

7 COMMON CAUSES AND FIXES

Cause	What it looks like	How to fix
AZ failure	All resources in one AZ down	Use multi-AZ and spread resources
Region outage	Whole region unavailable	Implement multi-region with DNS failover
Replication lag	Data not in secondary region	Check replication, increase bandwidth
DNS misconfiguration	Traffic still to failed region	Fix health checks and routing policy
Single-region dependency	A key service not replicated	Use multi-region or resilient alternative
RTO/RPO not defined	Slow recovery and data loss	Define and test RTO and RPO
Insufficient testing	Failover does not work	Regularly test failover and DR drills
Cascading failures	Multiple services impacted	Add isolation, circuit breakers, fallbacks

8 EXAMPLE: CHECK RDS FAILOVER READINESS

```
# Check RDS instance and replica status
aws rds describe-db-instances \
--db-instance-identifier mydb \
--query 'DBInstances[*].[DBInstanceIdentifier, DBInstanceStatus, SecondaryAvailabilityZone, ReadReplicaDBInstanceIdentifiers]'

# Initiate a failover (if Multi-AZ)
aws rds failover-db-cluster \
--db-cluster-identifier mydb-cluster

# Check afterwards
aws rds describe-db-instances \
--db-instance-identifier mydb
```

9 TROUBLESHOOTING STEPS

- 1 Confirm the scope (AZ or entire region)
- 2 Check cloud provider status and health events
- 3 Verify DNS and routing configuration
- 4 Check replication and data consistency
- 5 Validate dependent services and integrations

- 6 Test failover to secondary region
- 7 Check application logs and error rates
- 8 Identify single-region dependencies
- 9 Implement or use degraded operation mode
- 10 Communicate status and next steps

10 BEST PRACTICES

- Use multi-AZ for critical services
- Implement multi-region for high availability
- Define and test RTO and RPO
- Automate failover where possible
- Use health checks and DNS failover
- Keep dependencies cross-region or resilient
- Regularly run disaster recovery drills

11 USEFUL TOOLS AND LOGS

- AWS Health Dashboard
- Azure Service Health
- GCP Service Health
- Route 53 / Traffic Manager / Cloud DNS
- CloudWatch / Azure Monitor / Cloud Monitoring
- VPC Flow Logs / Network Watcher
- Application and database logs

★ SENIOR ENGINEER TIP

Design for failure, not just for scale. Regularly test failover, validate RTO/RPO, and ensure all critical dependencies can operate in a degraded mode.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Major Incident: End-to-End Investigation

Full production scenario from alert → triage → mitigation → RCA → prevention

1 COMMON SYMPTOMS

- Service is slow or completely down
- High error rates (5xx)
- Increased latency and timeouts
- CPU or memory spikes
- Database connection failures
- Queue backlogs
- Dependency failures (API, external service)
- User reports and business impact
- Multiple services affected
- Alarms firing across different systems

2 INCIDENT LIFECYCLE (END-TO-END FLOW)



The goal is to quickly restore service, understand what happened, and prevent it from happening again.

3 ALERT AND INITIAL TRIAGE

```

# Check alerts and system status
# Example (Prometheus)
curl -s 'http://prometheus:9090/api/v1/alerts?status=firing'

# Check service health
kubectl get pods -n <namespace>
kubectl get svc -n <namespace>

# Check recent changes
kubectl get deploy -n <namespace> \
  --sort-by=.metadata.creationTimestamp
  
```

Look for:

- What triggered the alert?
- Which services are affected?
- When did it start?
- Is this a new issue or ongoing?
- Recent deployments or changes?
- Scope of impact (users, regions, dependencies)

4 DEEP INVESTIGATION

```

# Check logs
kubectl logs <pod> -n <namespace>

# Check metrics
kubectl top pods -n <namespace>

# Check events
kubectl get events -n <namespace> \
  --sort-by=.lastTimestamp

# Check dependencies
curl -v https://<dependency-api>

# Check database
SELECT * FROM pg_stat_activity;
  
```

Look for:

- Error patterns in logs
- Resource exhaustion
- Slow queries
- Failed external API calls
- Network connectivity issues
- Dependency failures
- Configuration issues
- Correlation across services
- Evidence of the root cause

5 MITIGATION (RESTORE SERVICE)

```

# Example: Rollback deployment
kubectl rollout undo deployment/<name> \
  -n <namespace>

# Scale up if needed
kubectl scale deployment/<name> \
  --replicas=5 -n <namespace>

# Restart service
kubectl rollout restart deployment/<name>
  
```

Look for:

- Stop the immediate impact
- Rollback recent changes
- Scale resources (pods, nodes, databases)
- Failover to healthy region or AZ
- Disable non-critical features
- Use feature flags
- Clear queues or retry failed jobs
- Communicate status to users and stakeholders

6 ROOT CAUSE ANALYSIS (RCA)

```

# Build a timeline
# Identify the triggering event
# Find the root technical cause
# Identify contributing factors
# Analyze why detection was late
# Review what went well and what did not
  
```

Look for:

- What exactly caused the incident?
- Why did it happen?
- What allowed it to impact users?
- Were there missing alerts or gaps?
- Human or process factors?
- Could it have been prevented?

7 COMMON CAUSES AND FIXES

Cause	What it looks like	How to fix
Bad deployment	Errors after release	Rollback, fix and redeploy
Resource exhaustion	High CPU or memory	Scale up, optimize, set limits
Database issue	Slow queries or connection errors	Optimize query, increase connection pool, scale database
Dependency failure	External API timeouts	Retry with backoff, failover, use circuit breaker
Configuration change	Wrong settings	Revert change, validate configs
Network issue	Cannot reach service	Fix routing, security rules, DNS
Queue backlog	Messages piling up	Scale consumers, fix processing issue
Bug in application	Unhandled exceptions	Fix code, add error handling
Third party outage	External service down	Use fallback, degraded mode

8 PREVENTION AND IMPROVEMENT

- ✓ Add or improve monitoring and alerting
- ✓ Implement automated rollback
- ✓ Use canary or blue green deployments
- ✓ Set resource limits and autoscaling
- ✓ Add circuit breakers and retries
- ✓ Improve dependency health checks
- ✓ Run regular failover and disaster recovery drills
- ✓ Document runbooks and update incident response process
- ✓ Perform blameless post-incident review
- ✓ Address technical debt and configuration risks

9 INCIDENT INVESTIGATION CHECKLIST

- ✓ Confirm the problem and impact
- ✓ Check alerts, logs and metrics
- ✓ Identify recent changes
- ✓ Check service and dependency health
- ✓ Find the root cause
- ✓ Implement mitigation
- ✓ Verify service recovery
- ✓ Communicate status
- ✓ Document findings
- ✓ Create action items to prevent recurrence

10 KEY METRICS TO CHECK

- ✓ Error rate (5xx)
- ✓ Latency (p95, p99)
- ✓ CPU and memory usage
- ✓ Request rate and traffic patterns
- ✓ Database connections and query time
- ✓ Queue depth and consumer lag

11 USEFUL TOOLS AND LOGS

- ✓ Prometheus and Grafana
- ✓ Kubernetes (kubectl)
- ✓ CloudWatch / Azure Monitor / GCP
- ✓ Application logs (ELK, Loki, etc.)
- ✓ Tracing tools (Jaeger, K-Ray)
- ✓ Service health dashboards

★ SENIOR ENGINEER TIP

During a major incident, focus on restoring service first, then conduct a thorough RCA and implement preventive measures to reduce the chances and impact of future incidents.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta